

第十九届全国大学生信息安全竞赛（作品赛）
暨第三届“长城杯”网数智安全大赛（作品赛）
作品报告

☐ 命题赛道

☒ 自由赛道

作品名称：密捷：基于 ViT 动态剪枝的双向隐私推理系统

电子邮箱：3357024597@qq.com

提交日期：2026 年 7 月 4 日

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用A4纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

目录

摘要	7
第一章作品概述	12
1.1 背景与目标	12
1.2 基础知识	12
1.2.1 秘密共享与安全多方计算基础	12
1.2.2 混合表示与共享转换基础	13
1.2.3 Beaver 预处理与在线计算基础	14
1.2.4 Vision Transformer 基础	15
1.2.5 安全推理中的主要开销来源	16
1.3 核心贡献	17
1.4 相关工作与技术定位	18
1.4.1 通用私有推理系统演进	18
1.4.2 固定结构 Transformer 私有推理路线	19
1.4.3 动态词元优化与本作品定位	20
1.5 应用场景与交付边界	21
第二章系统设计	22
2.1 总体架构：离线训练部署阶段与在线安全推理阶段	22
2.2 总体软件流程	23
2.3 离线训练与部署准备流程	24
2.4 在线双向隐私推理流程	24
2.5 威胁模型与安全边界	25
2.6 DynamicViT 密态改写与固定图部署语义	28
2.7 硬件部署与软件基准	29
2.8 展示样例与真实部署框架	30
第三章系统实现	32
3.1 离线训练与部署准备实现	32
3.1.1 ABY 风格混合表示与协议适配	34
3.1.2 面向安全推理的函数改造策略	35
3.1.3 固定图安全化重写	38
3.1.4 执行结构与边界约束	40
3.2 在线两方安全推理流程与模块协同	42
3.3 前端控制面实现	42
3.4 服务端权威快检与审计	44
第四章测试方案与结果分析	48
4.1 测试环境、数据与评价指标	48
4.2 离线训练与部署准备实验	48
4.2.1 蒸馏补偿配对实验	49

4.2.2 部署对齐控制实验	49
4.2.3 base_rate 扫描与最终部署口径	50
4.2.4 阈值搜索与离线校准链路	51
4.3 基线对比	51
4.4 医疗主要交付场景结果	52
4.5 性能与通信量分析	54
4.5.1 代价分解与代理基准	55
4.5.2 安全比较网络的复杂度与通信量推导	58
4.5.3 端到端瓶颈来源	60
4.6 金融边界压力验证结果	60
4.7 协议层异常输入与鲁棒性验证	61
4.8 结果分析	63
4.8.1 结果结论与工程意义	63
4.8.2 对比口径与评价边界	63
4.9 消融实验与必要性验证	63
4.9.1 动态剪枝必要性	63
4.9.2 安全友好函数联合消融	64
第五章创新性与局限性	66
5.1 算法层创新	66
5.2 系统层创新	67
5.3 工程层特色与可验证交付能力	68
5.4 与常见方案对比	70
5.5 当前局限性	71
5.5.1 方法与实验边界	71
5.5.2 路线扩展与后续比较边界	72
第六章复现与参赛声明	74
6.1 最低可复现路径	74
6.2 数据、模型与权重合规声明	75
6.3 第三方许可、原创边界与技术公开范围	75
参考文献	78
附录 A 关键代码实现	82
附录 B 鲁棒性验证	85
附录 C 前端展示	87

表目录

表 2-1 系统威胁模型与安全边界定义.....	25
表 2-2 系统各维度安全边界覆盖情况.....	26
表 2-3 DynamicViT 原始操作与密态语义改写对照表.....	28
表 2-4 原型系统硬件部署与软件环境基准.....	30
表 4-1 测试数据集与评价指标说明.....	48
表 4-2 蒸馏补偿配对实验.....	49
表 4-3 secure_static_train_depth 部署对齐配对实验.....	50
表 4-4 阈值搜索与离线校准链路结果.....	51
表 4-5 医疗主要交付场景核心指标结果.....	53
表 4-6 算子级代理基准对比结果.....	55
表 4-7 代理路径比较结果.....	56
表 4-8 金融边界压力验证核心指标.....	61
表 4-9 协议层与控制面鲁棒性验证统计.....	62
表 4-10 安全友好函数联合消融.....	65
表 5-1 本作品与常见隐私推理方案能力对比.....	70
表 6-1 系统最低可复现验证步骤.....	74
表 6-2 数据、模型与权重合规边界说明.....	75
表 6-3 第三方组件许可与原创边界物理映射.....	76
表 B-1 各类异常场景的攻击特征与系统拦截层级明细.....	85
表 B-2 异常场景的系统返回处置、资源状态与证据来源明细.....	86

图目录

图 2-1 密捷双阶段协同流程图.....	22
图 2-2 端到端软件流转时序图.....	23
图 2-3 在线两方密态推理角色与数据流示.....	25
图 2-4 DynamicViT 密态改写前后对照图.....	29
图 2-5 系统物理与逻辑部署图.....	31
图 3-1 密捷项目当前框架总体架构图。.....	41
图 3-2 浏览器主线程与工作线程协同图.....	44
图 3-3 服务端权威快检与审计闭环图.....	46
图 4-1 base_rate 扫描结果与最终部署口径.....	50
图 4-2 不同方案医疗任务性能基线对比.....	52
图 4-3 动态安全剪枝与 DenseNet121 概率分布对比.....	54
图 4-4 不同场景端到端性能与通信量统计.....	54
图 4-5 各安全函数不同网络敏感性与单次安全执行的平均通信量.....	56
图 4-6 不同基础原语测试结果对比.....	57
图 4-7 协议层异常输入与控制面兜底验证矩阵.....	62
图 4-8 动态剪枝消融结果（524 张验证集，同口径）.....	64
图 5-1 本作品与常见方案能力边界对比图.....	71
图 C-1 管理员端登录与训练服务器连接配置界面.....	87
图 C-2 管理员端总览看板界面.....	88
图 C-3 管理员端训练任务创建界面.....	88
图 C-4 管理员端任务详情与日志界面.....	89
图 C-5 管理员端模型资产界面.....	89
图 C-6 管理员端结果证据界面.....	90
图 C-7 用户端首页界面.....	90
图 C-8 作品剪枝与执行概览界面.....	91
图 C-9 用户端隐私推理创建界面.....	91
图 C-10 用户端推理进度展示界面.....	92
图 C-11 用户端辅助诊断报告详情界面.....	92

摘要

人工智能视觉模型在医疗影像辅助诊断、金融风险识别、政务审核与智能安防等场景中得到广泛应用，对提升识别效率、降低人工审核成本和增强决策支持能力具有重要意义。当前主流视觉智能服务通常采用云端集中式推理模式，要求用户将原始图像、业务数据或中间特征上传至服务器完成模型预测。这一模式在提升使用便利性的同时，也带来了用户隐私泄露、模型资产暴露和中间状态被推断的风险，尤其在医疗、金融等高敏感场景中，已成为人工智能落地应用亟需解决的问题。

尽管安全多方计算、秘密共享和同态加密等隐私计算技术为安全推理提供了可行路径，但在 Vision Transformer (ViT) 等复杂视觉模型上仍面临多重挑战：（1）现有 ViT 隐私推理方案相比明文推理仍然效率低下，时延高；（2）目前的方案往往更关注单个协议或算子优化，缺少从模型训练、部署语义改写、密态推理到工程交付验证的完整闭环，限制了其在实际场景中的可复现性与可部署性。

为突破上述瓶颈，本作品设计并实现了一套**基于秘密共享的轻量级隐私保护 ViT 推理方案——密捷**。该系统围绕“训练期动态语义保留、部署期固定图改写、推理期双向隐私执行”的思路，构建了从离线模型准备到在线两方安全推理的完整流程，在隐私保护、推理效率与工程可交付性之间实现了较好的平衡，具备如下优势：

（1）**离线模型蒸馏与剪枝**：系统在训练阶段引入**教师-学生蒸馏**和**部署对齐约束**，针对 DynamicViT 明文动态剪枝难以直接迁移到密态执行的问题，将删除式 token 剪枝语义改写为固定图可执行的**掩码化表达**，减少训练语义与安全执行语义之间的偏差，在保证模型精度的同时降低密态推理的计算与通信负担。

（2）**在线隐私推理效率优化**：系统针对 ViT 密态推理中高代价算子进行**安全友好化改造**，以 fixed_square 激活、uniform attention 和 public-calibrated LayerNorm 等轻量化算子替代原始复杂路径；同时采用 BOLE、interleave、SIMD 等技术降低 Softmax、GELU 和归一化操作在秘密共享环境下的通信开销、加快计算速度，有效提升密态推理效率。

（3）**工程闭环完整**：在以上基础上，系统不仅包含在线两方密态推理过程，还覆盖离线训练准备、阈值搜索、模型冻结、bundle 导出、客户端本地分片、服务端权威快检和审计证据记录等环节，并构建可视化、易操作的模型使用平台，形成**可复现、**

可验证、可迁移的作品交付链路，实现工程场景完整闭环。

密捷系统主要包含五个组成部分：**离线模型训练与蒸馏模块、安全友好算子改写模块、固定图部署语义转换模块、两方秘密共享推理模块以及前后端交互与审计验证模块**。实验结果表明，系统在保持基本任务可用性的前提下，能够有效降低关键安全算子的通信与执行代价；在同形代理基准中，当前改写路径相较传统 ReLU + Softmax 路径实现了约 46.62% 的时延下降和约 85.11% 的通信量下降。进一步的 local/LAN/WAN 单函数测试与共享转换分析表明，系统优化重点能够有效针对安全推理中的主要通信瓶颈。

本作品提出了一套兼具隐私保护能力、轻量化推理效率和工程交付完整性的**视觉模型安全推理方案**，适用于医疗影像辅助诊断、金融风险识别等对数据安全要求较高的实际应用场景，为隐私保护视觉智能服务的落地提供了可行参考。

关键字：隐私保护推理 秘密共享 Vision Transformer DynamicViT 知识蒸馏
安全多方计算

Abstract

Artificial intelligence visual models are widely employed in scenarios such as medical imaging-assisted diagnosis, financial risk identification, government review processes, and intelligent security systems, playing a crucial role in enhancing recognition efficiency, reducing manual review costs, and strengthening decision support capabilities. Current mainstream visual AI services typically adopt a cloud-based centralized inference model, requiring users to upload raw images, business data, or intermediate features to servers for model prediction. While this approach improves user convenience, it also poses risks including user privacy breaches, exposure of model assets, and inference of intermediate states—issues that have become critical challenges in implementing AI applications, particularly in highly sensitive domains like healthcare and finance.

While privacy-preserving computing technologies such as secure multi-party computation, secret sharing, and homomorphic encryption offer viable approaches for secure inference, significant challenges remain in complex visual models like Vision Transformers (ViT): (1) Existing ViT privacy-inference solutions remain less efficient and exhibit higher latency compared to plaintext inference; (2) Current approaches often focus solely on optimizing individual protocols or operators, lacking a comprehensive end-to-end framework that spans model training, semantic rewriting during deployment, confidential inference, and engineering validation—thereby limiting their reproducibility and deployability in practical applications.

To overcome the aforementioned limitations, this work designs and implements a lightweight privacy-preserving ViT inference framework named **MiJie**, based on secret sharing. The system adopts the approach of "dynamic semantic preservation during training, fixed graph rewriting during deployment, and bidirectional privacy enforcement during inference," establishing a comprehensive workflow from offline model preparation to secure two-party inference online. It achieves an optimal balance between privacy protection, inference efficiency, and engineering deliverability, offering the following advantages:

- (1) **Offline model distillation and pruning:** During training, the system incorporates

teacher-student distillation and **deployment alignment constraints**. To address the challenge of directly transferring DynamicViT's plaintext-based dynamic pruning to encrypted execution, it transforms deletional token pruning semantics into mask-based representations executable on fixed graphs, thereby reducing the discrepancy between training semantics and secure execution semantics while maintaining model accuracy and alleviating computational and communication costs in encrypted inference.

(2) **Optimization of Online Privacy-Inference Efficiency**: The system enhances **security compliance** for high-cost operators in ViT Secret State Inference by replacing complex original paths with lightweight alternatives such as `fixed_square` activation, uniform attention, and publicly calibrated LayerNorm. Additionally, it employs techniques like BOLE, interleave, and SIMD to reduce communication overhead associated with Softmax, GELU, and normalization operations in privacy-preserving environments, thereby accelerating computation speed and significantly improving privacy-preserving inference efficiency.

(3) **Comprehensive engineering closed-loop**: Building upon the aforementioned foundation, the system not only encompasses online two-party secure reasoning processes but also covers all stages including offline training preparation, threshold search, model freezing, bundle export, client-side local partitioning, server-side authoritative fast verification, and audit evidence recording. It further establishes a visual and user-friendly model utilization platform, creating a **reproducible, verifiable, and transferable** product delivery pipeline that achieves a fully integrated engineering workflow.

The MiJie system primarily consists of five components: **an offline model training and distillation module**, **a secure and user-friendly operator rewriting module**, **a semantic transformation module for fixed graph deployment**, **a two-party secret-sharing inference module**, and **a front-end back-end interaction and audit verification module**. Experimental results demonstrate that the system effectively reduces the communication and execution costs associated with critical security operators while maintaining essential task functionality. In the isomorphic proxy benchmark, the proposed rewriting approach achieves approximately 46.62% reduction in latency and 85.11% decrease in communication overhead compared to the traditional ReLU + Softmax path.

Further tests using local, LAN, and WAN single-function scenarios, along with shared transformation analysis, confirm that the system's optimizations precisely address the primary communication bottlenecks in secure inference.

This work proposes **a visual model-based secure inference framework that combines robust privacy protection, efficient lightweight inference, and comprehensive engineering deliverability**. It is suitable for practical applications requiring stringent data security—such as medical imaging-assisted diagnosis and financial risk identification—and provides a viable reference for implementing privacy-preserving visual intelligence services.

Keywords: Privacy protection Inference Secret sharing Vision Transformer

DynamicViT Knowledge distillation multi-party computation

第一章作品概述

1.1 背景与目标

人工智能模型正加速进入医疗、金融、政务等高敏感场景，但这些场景中的输入数据、模型参数和中间推理状态往往同时具有业务敏感性与隐私属性。若直接采用明文推理，系统即使易部署，也会暴露原始数据和模型资产；若直接套用通用安全计算协议，又容易因高代价非线性、归一化和动态控制流带来过高通信与时延。

在这一背景下，基于秘密共享（Secret Sharing, SS）[1]的安全多方计算（Secure Multi-Party Computation, MPC）[2,3]因能够较自然支持张量加法、乘法与矩阵运算，成为隐私保护推理的重要路线之一。然而，Vision Transformer（ViT）[5]在安全场景中的落地仍然困难。其核心推理过程包含大规模注意力矩阵计算，以及 Softmax、GELU[7]、LayerNorm [8]等高代价算子；当这些步骤进入安全两方计算（Two-Party Computation, 2PC）环境后，通信量、交互轮次与共享表示转换成本都会显著上升，局域网和广域网环境下尤为明显。

并且，动态视觉 Transformer（Dynamic Vision Transformer, DynamicViT）[9]等动态结构模型还引入了另一层难点：词元保留、排序与删除本质上依赖输入相关路径选择。若直接在安全边界外完成这些操作，会泄露动态路径信息；若完整照搬原始明文逻辑进入安全环境，又会使比较、排序和布尔/算术域转换开销迅速增加。

于是本作品聚焦于秘密共享约束下的轻量级 ViT 隐私保护推理问题，目标是在不泄露用户输入、模型参数和关键中间状态的前提下，使动态 ViT 形成可部署、可验证、可复现的推理闭环。本作品关注的核心问题是如何围绕真实瓶颈对高代价算子、动态词元路径和部署边界进行协同改写，从而在隐私保护条件下获得可用精度、可控通信与可交付性能。

1.2 基础知识

为说明本作品的设计依据，本节简要介绍秘密共享、安全多方计算、混合表示、预处理材料和 Vision Transformer 中与安全推理开销直接相关的基础概念。

1.2.1 秘密共享与安全多方计算基础

安全两方计算可追溯到 Yao 提出的安全计算协议[2]，通用安全多方计算理论则可由 GMW 框架[3]支撑。秘密共享是安全多方计算中的核心数据表示方式之一，其基本思

想是将一个明文秘密拆分为若干份随机共享，并分别分发给不同参与方，使得任意单方仅持有局部信息，无法独立恢复原始数据；只有在满足协议约束时，各参与方才能联合得到最终结果。与直接传输明文不同的是，秘密共享能够在不泄露输入数据、模型参数或中间特征的前提下完成协同计算，因此被广泛用于隐私保护推理场景。

在最基本的加法型秘密共享中，设明文为 x ，两方可随机采样一个共享值 x_1 ，并令另一共享值满足

$$x_2 = x - x_1(\text{mod } p),$$

其中 (p) 表示有限域模数。此时有

$$x = x_1 + x_2(\text{mod } p).$$

参与方分别持有 x_1 与 x_2 ，单独观察任一共享值都不能恢复 x ，但两方相加即可重建秘密。这种思想可以自然推广到张量和矩阵，所以适合神经网络中的线性层、卷积层和残差连接等张量运算。

从计算特性看，秘密共享环境对不同类型的算子支持能力并不均衡。加法、减法和线性组合通常可以局部完成，代价较低；而乘法、比较、除法、开方及复杂非线性则需要额外的协议交互或预处理材料支持，所以代价较高。对于神经网络推理，真正制约性能的是激活函数、归一化、注意力分布计算和比较相关操作。简单来讲，优化安全推理系统，保证模型正常运行只是基础，重要的是找出共享域内运算开销过大的核心算子并针对性重构。

在本作品所关注的隐私保护 ViT 推理场景中，客户端输入通常以秘密共享形式进入计算链路，服务端或协作节点在不接触明文图像的条件下完成模型前向。整个推理过程可以理解为：明文输入先被拆分为共享形式，然后经过若干安全张量运算，最后仅在输出阶段恢复类别分数或判定结果。故而秘密共享不仅仅是一个底层协议组件，更决定了后续所有函数改造和性能分析的基本边界。

1.2.2 混合表示与共享转换基础

在安全多方计算中，单一共享表示通常难以同时兼顾所有算子的实现效率。为此，研究者提出了混合表示思想，即针对不同操作类型采用不同共享形式，并在必要时进行表示切换。最常见的两类表示是算术共享（Arithmetic Sharing, AS）和布尔共享（Boolean Sharing, BS）。

算术共享更适合处理加法、乘法、矩阵乘法和线性投影等数值计算，因此在神经

网络主干计算中应用最广。布尔共享则更适合处理按位逻辑、比较、符号位提取、最大值选择和阈值判断等操作。由于 Vision Transformer 中既包含大量线性层，也包含 softmax、裁剪、阈值判断和部分非线性子过程，导致仅依赖单一共享形式往往会致代价失衡。特别是当某些算子在算术域中需要模拟比较逻辑时，通常会引入较高的通信和交互成本。

为了解决这一问题，混合协议通常引入两类基础转换原语：A2B（Arithmetic-to-Binary）和 B2A（Binary-to-Arithmetic）。A2B 用于将算术共享表示转换为布尔共享表示，适合在进入比较或逻辑处理阶段前使用；B2A 则在布尔域操作完成后，将结果重新映射回算术域，以便继续参与后续乘法或线性组合。其抽象流程可写为

$$[x]_A \xrightarrow{A2B} [x]_B \xrightarrow{\text{Compare/Logic}} [b]_B \xrightarrow{B2A} [b]_A$$

A2B/B2A 的存在使系统能够按需调用更适合当前任务的共享表示，但这类转换本身并非零代价。特别是在跨网络环境中，表示转换往往需要额外通信、同步或布尔域原语支持，因此会成为重要性能瓶颈。通俗来讲，混合表示的核心优势不在于频繁执行格式转换，在于仅在确有需求的节点完成转换操作。从系统设计角度看，一个高效的安全推理方案，是尽量让大部分线性张量运算保留在算术域中，将布尔域限定在比较、符号位和局部逻辑判断等不可避免的环节，不是频繁在算术域与布尔域之间往返。

本作品与混合表示技术的关系也应如此理解。本作品没有将 ABY 2.0[43]或其他混合协议框架作为完整推理主协议，而是吸收其核心思想，在局部子过程中保留 A2B/B2A 风格的共享转换机制，用于支撑比较与部分非线性相关步骤。由此，在本作品的方法设计中，A2B/B2A 更接近底层协议适配工具。

1.2.3 Beaver 预处理与在线计算基础

在秘密共享环境下，乘法通常比加法昂贵得多。为了降低在线阶段的交互负担，许多安全多方计算协议会采用“离线预处理+在线计算”的两阶段设计。其中，Beaver triples[11]是最经典的预处理材料之一。其基本思想是预先生成随机三元组 (a, b, c) ，满足 $c = a \cdot b$ ，并将三者分别以秘密共享形式分发给参与方。这样，在在线阶段需要计算两个秘密共享值 x 与 y 的乘积时，就可以借助三元组把原本复杂的乘法过程转化

为更简单的线性组合与少量打开操作，从而降低实时交互开销。

从抽象角度看，Beaver triples 的价值在于把“与输入无关的随机相关性生成”提前到离线阶段完成，而把真正依赖用户输入的数据交互压缩到在线阶段。类似地，在布尔域乘法、随机比特生成以及某些 B2A 相关步骤中，也常常需要额外的预处理材料或随机相关性支持。因此，在安全推理系统中，预处理材料的生成、分发和调用方式，会直接影响协议的吞吐能力和部署稳定性。

然而，需要注意的是，不同系统对预处理材料的实现方式并不完全相同。有些框架会显式统计离线阶段的通信和存储代价，有些框架则通过伪随机相关性生成、可信初始化方或本地相关性机制弱化在线统计中的离线痕迹。因此，“离线代价是否为零”必须结合具体 provider、共享机制和测量口径理解。

对本作品，Beaver triples 和相关预处理材料的重要性主要体现在两个方面。第一，它们说明了为什么乘法和部分转换可以在理论上被拆分为“预处理”与“在线执行”两部分；第二，它们也帮助解释了实验中观察到的现象，即某些材料的在线统计通信量并不显著，而真正的瓶颈常常集中在共享表示转换和高代价非线性上。所以本作品在后续实验中不仅关注了高层算子时延，也关注预处理材料与转换原语的开销分解。

1.2.4 Vision Transformer 基础

Vision Transformer (ViT) 是一类将 Transformer 引入视觉任务的模型结构，其核心自注意力机制 (Self-Attention) [6] 来源于 Transformer 架构。核心思想是将输入图像划分为若干固定大小的 patch，并将每个 patch 映射为一个 token，再利用多层 Transformer 编码器对 token 序列进行全局建模。设输入图像被划分为 N 个 patch，则经过线性嵌入后可表示为

$$Z_0 = [x_{\text{cls}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}},$$

其中 x_{cls} 表示分类 token， x_p^i 表示第 i 个 patch， E 表示 patch embedding 线性映射， E_{pos} 表示位置编码。这样，二维图像被转换为一维 token 序列，后续可以直接输入 Transformer 模块处理。

在 Transformer 编码器内部，自注意力是最核心的计算单元。对于给定输入特征，通常先通过线性变换获得 Query、Key 和 Value 三个矩阵 Q 、 K 、 V ，再计算注意力输出。对于 Vision Transformer 路线，其核心注意力计算为公式 (1-1)。

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1-1)$$

其中 d_k 为 Key 向量维度。该公式说明，自注意力首先通过 QK^T 衡量 token 间相关性，再通过 softmax 将相关性归一化为权重分布，最后对 V 做加权求和。多头注意力机制则在多个子空间中并行执行上述过程，从而提高模型对不同关系模式的表达能力。

除注意力模块外，ViT 还包含前馈网络 MLP 和归一化层。一个典型 Transformer block 通常可表示为

$$\begin{aligned} Z'_l &= \text{MSA}(\text{LN}(Z_{l-1})) + Z_{l-1}, \\ Z_l &= \text{MLP}(\text{LN}(Z'_l)) + Z'_l, \end{aligned}$$

其中 MSA 表示多头自注意力（Multi-Head Self-Attention），LN 表示层归一化（Layer Normalization），MLP 表示前馈网络（Multi-Layer Perceptron）。可以看到，ViT 的主干虽然由矩阵乘法和残差连接构成，但注意力中的 softmax、MLP 中的激活函数以及归一化中的方差与开方计算，都会在安全环境下引入高代价非线性。

从安全推理视角看，ViT 相比传统 CNN 有两个突出特点。其一，注意力矩阵规模通常随 token 数平方增长，这会显著放大通信和内存压力；其二，ViT 对 softmax、GELU 和 LayerNorm 依赖较强，而这些算子恰好是秘密共享中最难高效实现的部分。因此，ViT 的安全化不能只依赖底层协议替换，还必须针对注意力、激活和归一化路径进行结构性改造。这也是本作品后续重点围绕 fixed_square、uniform attention 和校准归一化展开设计的直接原因。

1.2.5 安全推理中的主要开销来源

在明文环境中，模型效率通常主要由参数量、浮点运算次数和显存占用衡量；而在安全推理环境中，性能评估维度会明显扩展。除本地计算量外，通信量、交互轮次、共享表示转换次数以及网络时延敏感性，都会直接影响端到端推理性能。因此，一个在明文中计算便宜的算子，在秘密共享环境下未必同样高效；反之，一些明文中较粗糙的低阶近似，在安全环境中可能因为显著减少通信而更具优势。

从算子层面看，安全推理中的主要开销通常集中在四类操作上。第一类是高代价非线性，如 GELU、Sigmoid、Tanh 等，它们往往依赖高阶近似、比较或额外乘法深度。第二类是注意力归一化，尤其是 Softmax，其包含指数、求和和除法，在 token

维度较大时会形成显著瓶颈。第三类是归一化算子，如 LayerNorm，其中均值、方差、平方和倒数平方根等步骤在秘密域中实现成本较高。第四类是共享表示转换及相关比较操作，即 A2B/B2A、最高位提取、阈值判断等，它们对网络环境高度敏感，在跨节点场景下常常成为决定性因素。

因此，安全推理系统的优化重点不应仅停留在“减少参数量”或“减少浮点运算”，而应进一步考虑“减少高代价协议原语的触发次数”。这意味着模型结构改造、函数近似和协议适配必须联合考虑。例如，若某种改造形式能够减少 softmax 调用次数、降低激活函数复杂度并减少 A2B/B2A 往返，即便它在明文精度上不是最优，也可能在安全部署中获得更好的整体收益。

本作品正是在这一认识基础上展开后续设计。通过把固定平方激活、均匀注意力和校准归一化作为核心安全友好算子，并结合局部混合表示适配机制，本作品试图将 ViT 中最昂贵的几个步骤重写为更适合秘密共享执行的形式。后续方法章节将详细说明这种改造是如何具体落到激活函数、注意力路径和归一化实现上的。

1.3 核心贡献

围绕高敏感视觉任务中“输入与模型隐私保护”与“动态 Transformer 可部署推理”之间的矛盾，本作品在两方半诚实威胁模型下构建隐私保护 ViT 推理方案，重点解决动态词元剪枝（dynamic token pruning）语义难以直接进入固定图安全执行环境的问题。核心贡献可归纳为算法层、系统层和交付层三方面：

（1）算法层：本作品将 DynamicViT 的输入相关删除式剪枝重写为固定图下的词元保留掩码（keep-mask）语义，并围绕激活、注意力与归一化三类高代价环节构建由 fixed_square、uniform attention 和 public-calibrated LayerNorm 组成的正式安全友好函数主线，使动态词元路径能够进入 2PC 场景并保留可校准的判别边界。

（2）系统层：本作品构建了“训练期保留动态语义、部署期冻结为 bundle、在线期执行两方安全推理”的三层实现路径，并以浏览器本地分片、服务端权威快检、异常输入拦截和重放/并发门控形成端到端执行闭环，使算法语义、部署语义与安全边界保持一致。

（3）交付层：本作品围绕离线训练、阈值搜索、冻结导出、通信量分解、联合消融与控制面证据留存建立了可复现的交付链路，使作品不仅能展示算法效果，还能以 bundle、脚本和审计文件的形式接受复核，并为后续扩展到更复杂场景或更高阶多

方设置提供基础。

1.4 相关工作与技术定位

隐私保护神经网络推理[12]的目标是在不暴露用户输入、模型参数及中间特征的前提下完成预测计算。围绕这一目标，现有研究主要形成了三类技术路线：基于同态加密（Homomorphic Encryption, HE）[4]的加密域推理、基于安全多方计算的协同推理，以及结合可信执行环境或混合协议的折中方案。不同路线在安全假设、通信模式、计算复杂度和工程可部署性方面各有侧重，其中安全多方计算，尤其是基于秘密共享的推理框架，由于能够较自然地支持张量加法、乘法和矩阵运算，已成为隐私保护深度学习中的重要研究方向。

早期隐私保护推理研究主要围绕卷积神经网络(Convolutional Neural Network, CNN)、全连接网络等结构展开，关注的核心问题是如何在保证安全性的同时降低非线性层、比较操作和通信开销。相关工作已经证明，线性层本身通常不是最难处理的部分，真正限制端到端效率的往往是激活函数、最大值选择、归一化、比较和除法等操作。这一认识推动研究重点逐步从“能否在密文或共享域中完成推理”转向“如何针对模型结构特征设计更低代价的安全执行路径”。

从整体趋势看，隐私保护推理的发展已经经历了从协议可行性验证到系统优化、再到模型结构协同设计的演进过程。也就是说，当前研究不再满足于在加密域或共享域中机械复现原始神经网络，而是越来越强调模型本身需要为安全计算环境做适配。本作品的工作正是在这一背景下展开：围绕其高代价算子、动态 token 路径和工程交付边界进行重构，以获得更适合安全推理的轻量化部署形式。

1.4.1 通用私有推理系统演进

从更长的研究脉络看，隐私保护推理最初并非围绕 Transformer 展开，而是先在卷积网络和一般前馈网络上建立问题定义、协议基础与系统接口。SecureML[13]较早系统化讨论了面向机器学习任务的秘密共享计算框架，说明了线性层、乘法和若干基础非线性在多方环境中的可实现性。MiniONN[14]进一步强调将已训练完成的神经网络转换为 oblivious neural network, 使既有模型在不重构训练流程的前提下进入私有推理；GAZELLE[15]则通过同态线性层与 garbled circuits 非线性层的混合设计，展示了“不同算子采用不同协议后端”的工程优势。随后，XONN[16]利用二值化网络与布尔电路进一步压缩线性层和激活层开销，推动研究从“保护现有模型”向“为安全推

理重塑模型形态”演进。

在这一阶段，Delphi[17]、CrypTen[18]等工作又进一步把研究重点从单个协议原语推进到通用推理系统和工程抽象。Delphi 更强调系统级推理性能与协议组合设计，CrypTen 则通过贴近深度学习框架的接口降低了安全推理原型构建门槛，使研究者能够在更高层面分析神经网络的密态执行代价。总的来看，这一时期的代表性成果已经较好解决了输入保护、通用张量接口、线性层安全执行以及部分非线性计算问题，初步建立了“从协议到系统”的私有推理技术底座。

但这类通用系统大多默认网络结构固定、控制流简单，中间执行路径与输入弱相关，因此其主要解决的是“如何在安全环境下完成神经网络计算”，而不是“如何让具有动态词元路径的 Transformer 在安全边界内稳定落地”。因此，通用私有推理系统为本作品提供了协议与接口基础，但尚未直接回答动态结构与固定图安全执行之间的兼容问题。

1.4.2 固定结构 Transformer 私有推理路线

当研究对象从卷积网络转向 Transformer 后，困难的焦点明显转移。相较卷积结构，Transformer 不仅包含更高维度的矩阵乘法、更频繁的归一化与激活计算，而且其推理开销会随 token 数量显著放大。公式(1-1)表明，Transformer 的关键不只在大规模矩阵乘 QK^T ，还在于随后引入的 Softmax 归一化；而在一个标准 Transformer block 中，GELU 激活和 LayerNorm 也会频繁出现。所以固定结构 Transformer 的私有推理难点不仅是“如何安全实现已有算子”，还是“如何让模型结构本身更适合密态执行”。

围绕这一问题，相关研究大致可以分为三类。第一类以 Iron[20]、BOLT[21]为代表，重点分析或优化 Transformer 中矩阵乘、Softmax、激活和归一化的复合代价，强调在交互式协议环境中降低非线性与通信成本。第二类以 BumbleBee[22]、THE-X[24]为代表，更强调通过协议组合、算子替换与近似函数设计压缩整体执行代价，说明 Transformer 私有推理不能简单照搬卷积网络时代的安全实现思路。第三类则以 NEXUS[23]、MPCFormer[25]、SecFormer[26]为代表，进一步关注交互轮次、算子形态和 SMPC 友好结构对在线性能的影响，尝试通过更低交互的函数近似、更可控的结构设计或更适合 MPC 的执行路径来改善部署效率。

这些工作的共同贡献在于，它们明确揭示了 Transformer 私有推理是一个独立问题，而不是卷积网络安全方案的简单平移。已有研究表明，Transformer 私有推理的真

正瓶颈往往集中在 Softmax、GELU、LayerNorm 以及与比较、除法和归一化相关的子过程上。同时，这些工作也表明，若只在底层协议上做优化，而不对模型中的高代价算子进行结构性改写，往往难以获得可部署的端到端收益。

不过，这一路线默认推理图基本固定，中间词元数量与执行路径不随输入显著变化，所以更适合标准 BERT、ViT 或固定长度序列场景。它为本作品提供了算子级和协议级优化启发，但仍不足以直接解决动态结构在安全环境中的部署问题。

1.4.3 动态词元优化与本作品定位

另一类与本作品关系更直接的研究来自明文高效视觉 Transformer 领域。DynamicViT 通过逐阶段词元打分和删除式稀疏化降低后续层计算量；EViT[27]通过保留高关注 token 并融合低关注 token 来减少冗余计算；A-ViT[28]借助自适应停止机制使不同 token 在不同深度提前退出；ToMe[29]则通过 token merging 在几乎不改训练流程的前提下提高吞吐；MPCViT[30]进一步把“模型结构是否适合 MPC 执行”显式纳入搜索目标。这些方法共同说明，视觉 Transformer 的效率问题与 token 数量管理高度相关，动态稀疏化、逐层筛选和结构重排可以显著改善推理成本。

学习可知，明文高效 ViT 方法通常不能无改动地迁移到私有推理场景。无论是删除 token、让 token 提前停止，还是合并相似 token，本质上都涉及数据相关路径选择、变长结构、importance 排序或显式比较判断。假设直接在安全执行域外部完成这些操作，就可能泄露输入分布、模型偏好甚至部分中间状态；如果完全照搬原始明文逻辑进入安全环境，又会使密态计算图难以固定，比较、排序、A2B/B2A 转换以及通信开销迅速上升。由此，动态 token 推理在私有推理场景中的难点，是“如何在不泄露路径信息的前提下完成可控剪枝”，而不是“如何剪枝”。

CipherPrune[21]开始把词元重要性差异直接纳入私有推理优化目标，通过密态渐进式剔除低重要性词元，并配合不同强度的多项式近似，同时缓解 token 数量与非线性算子带来的双重成本。这类工作带来的重要启发在于：在安全推理中，token 管理本身就可以成为独立优化对象，不能只把它视为模型层的附属技巧。与固定结构 Transformer 私有推理相比，这一路线更接近真实视觉模型的输入依赖特征，但同时也对协议设计、执行边界和工程交付提出了更高要求。

因此，动态词元优化路线说明了“为何要管理词元规模”，却没有自然给出“如何在不泄露路径信息的前提下完成安全执行”的答案。本作品的切入点正是在保留

DynamicViT 训练语义的同时，将其改写为可进入固定图 2PC 环境的密态表达。

总体而言，现有相关工作可概括为三条路线：通用私有推理系统主要解决基础协议与神经网络算子安全执行问题；固定结构 Transformer 私有推理路线主要降低注意力、激活和归一化等算子的密态执行成本；动态词元优化路线则关注如何减少视觉 Transformer 的冗余计算。本作品位于后两类问题的交叉处，重点将 DynamicViT 的输入相关剪枝语义重写为可进入固定图安全执行环境的密态表达。

1.5 应用场景与交付边界

本作品面向医院侧数据与模型提供方参数同时敏感的协同推理场景，目标是在不暴露原始影像和明文模型参数的前提下完成医疗影像分类。系统的主要交付对象包括终端用户、医院侧部署环境、服务端控制面以及两方安全执行域。终端侧负责本地解码、预处理、质量评估与秘密分享生成；服务端负责协议预检、重放与并发守卫、审计落盘和密态推理调度；安全执行域负责完成双向隐私约束下的前向计算并返回最终分类结果。

本作品的交付边界不包括生产级拒绝服务防护、恶意参与方串谋对抗、输出侧隐私泄露建模以及完整的多中心部署评估。本作品的重点是证明：在双向隐私约束下，动态剪枝语义能够被安全化重写，并形成可运行、可验证、可复现实验闭环。

对医疗影像任务而言，系统同时面对医院侧原始影像隐私、模型提供方参数保护和视觉 token 冗余三类约束。若完全退回固定结构模型，虽然协议实现更简单，但会削弱动态剪枝在时延和通信量上的潜在收益。因此，本作品关注的是在双向隐私、动态边界保留与工程闭环交付之间建立可验证的平衡。

第二章系统设计

2.1 总体架构：离线训练部署阶段与在线安全推理阶段

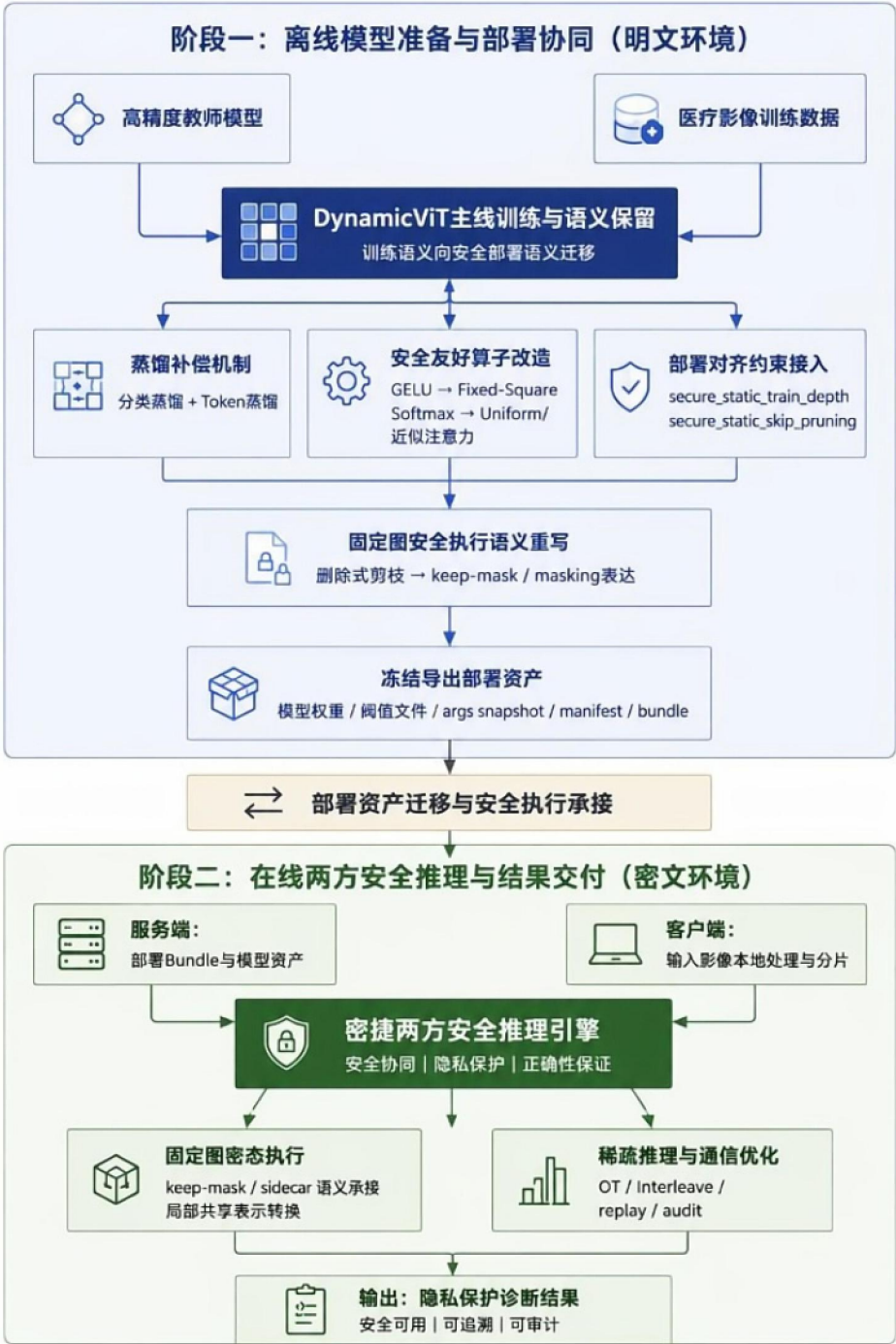


图 2-1 密捷双阶段协同流程图

由图 2-1 可见，方案整体流程分为两大阶段：离线阶段完成训练与部署的配套预

处理工作和在线阶段执行两方安全推理运算。前者负责保留 DynamicViT 的判别语义，并通过知识蒸馏（Knowledge Distillation, KD）[32][33]、安全友好算子注入、部署对齐约束和阈值校准，将原始明文模型重构为适合固定图安全执行的部署资产；后者则在两方秘密共享环境下承接这些冻结语义，完成隐私保护条件下的安全推理与结果返回，从而形成从模型训练到工程交付的完整闭环。

2.2 总体软件流程

图 2-2 给出了完整的软件流转时序。主线程仅负责本地影像加载、预览和请求编排；浏览器工作线程[39]独立完成图像解码、裁剪、数据质量评估指标提取、审计链构建与分片序列化；服务端首先执行原始报文体、边界参数、报文头与多部件结构校验，再对结构化元数据、密态分片与重构张量进行权威快检，全部通过后方可进入安全处理单元（Secure Processing Unit, SPU）[34]密态前向计算。最终结果回传时同时包含分类结论、质量裁决、审计摘要与控制面开销，使展示链路与安全链路保持一致。

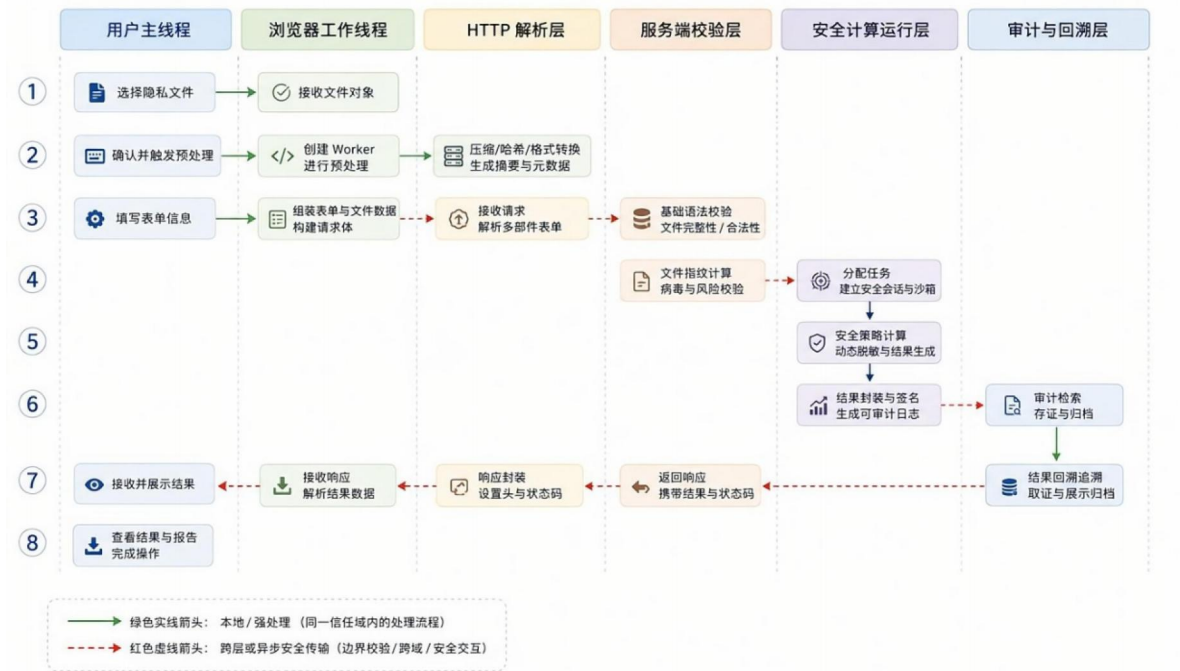


图 2-2 端到端软件流转时序图

需要强调的是，图 2-2 所示的软件时序是建立在离线训练与部署准备已经完成的前提之上。也就是说，正式进入在线阶段前，系统已经完成 DynamicViT 主线训练、教师-学生蒸馏补偿、安全友好算子替换、部署对齐约束、阈值搜索以及 bundle 冻结导出；在线链路读取的是一组带有配置文件、阈值口径和命令快照的正式部署资产。

这种“两阶段协同”设计的意义在于，把训练期可调语义收束为部署期可验证资产，再由浏览器端本地分片、服务端权威快检和两方安全执行链路统一承接。由此，本作品中的在线 2PC 推理是以前序模型准备和部署冻结为前置条件的完整交付闭环。

2.3 离线训练与部署准备流程

离线阶段的核心目标不仅仅是单纯获得较高的明文精度，更是让后续在线 2PC 主线能够稳定承接 DynamicViT 的判别语义。系统先以 DeiT-S 形状为骨架保留第 3、6、9 个 stage 的词元筛选位置，再通过教师-学生蒸馏维持动态 token 路线在裁剪后的特征稳定性；随后引入安全友好激活、近似注意力与 `secure_static_train_depth` 等部署对齐约束，使训练语义逐步向固定图安全执行口径收束。

在训练收敛后，系统并没有直接把检查点送入在线推理，而是继续执行阈值搜索、参数冻结和 bundle 导出。该步骤会把权重快照、阈值口径、配置文件、参数快照与命令记录统一固化为部署资产，使在线阶段读取到的是可核验、可复现、可审计的一组文件，而不是仅凭口头说明解释的模型状态。

2.4 在线双向隐私推理流程

在线阶段由浏览器本地预处理与秘密分享、跨边界密态传输、服务端权威快检以及两方安全执行域四部分组成。明文图像仅在终端工作线程内完成解码、裁剪、归一化、质量摘要与份额生成；跨域后只保留份额、结构化元数据和审计摘要。服务端在进入 SPU 前先完成协议结构、哈希关系、张量合法性和状态守卫校验，随后才触发 2PC 密态前向。

为了进一步明确在线阶段的角色边界，图 2-3 给出了密态推理系统的抽象数据流。用户侧仅在本地持有原始明文输入和最终结果恢复能力；模型持有方持有模型参数及相关密钥材料；计算辅助服务器提供协议执行、计算调度和协同计算支持。跨越信任边界传输的数据均以密文数据或秘密共享分片形式存在，任一非授权单方无法直接恢复用户输入、模型参数或关键中间特征。

因此，本作品中的在线链路严格承接离线阶段已经冻结的部署语义。离线阶段解决的是判别边界如何迁移到安全执行环境，在线阶段解决的是这些边界如何在双向隐私约束下被正确、稳定地执行。

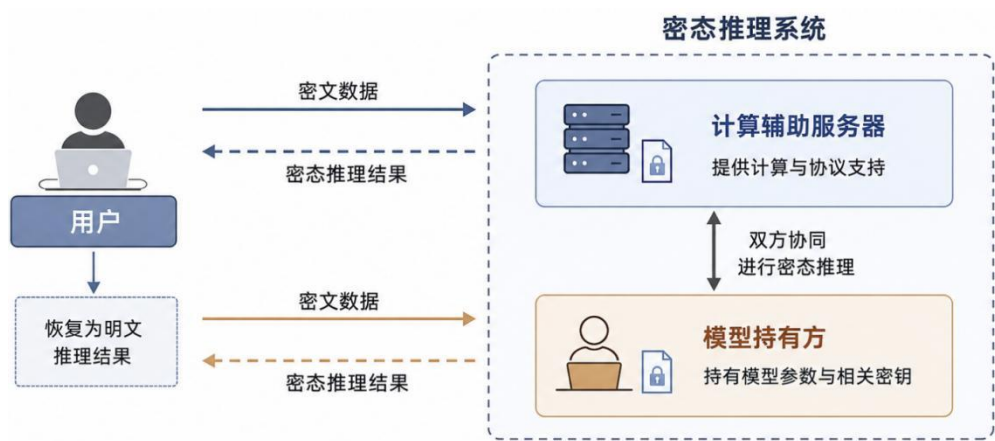


图 2-3 在线两方密态推理角色与数据流示意图

2.5 威胁模型与安全边界

表 2-1 系统威胁模型与安全边界定义

项目	说明
保护对象	用户输入数据、模型参数、中间动态决策状态、审计与控制面证据。
参与方	医院终端浏览器、浏览器工作线程、业务侧前置网关、服务端快检模块、两方 SPU 节点。
攻击者能力	可监听网络、构造畸形多部件表单报文（multipart/form-data）[37]、伪造结构化元数据、重放一次性随机标识（nonce）、提交异常密态分片、实施并发探测。
安全假设	两方半诚实；不引入额外可信第三方；前端不作为可信根，关键裁决以服务端复算为准。
不覆盖范围	恶意参与方串谋、浏览器被完全控制、进入长耗时 SPU 后的主动断连感知、生产级分布式 DoS、防输出反演攻击。
防护证围	协议层模糊测试、重放与并发守卫测试、服务端张量快检与审计落盘证据。

本作品采用两方半诚实威胁模型。系统中，医院终端持有原始输入数据，模型持有方与协同计算方分别持有安全执行域中的不同份额。参与方均遵循协议执行，但可能试图从交互过程中推断对方的隐私信息。系统假设两方不串谋，且不考虑恶意篡改协议、主动注入畸形中间值或侧信道攻击等更强对抗情形。保护的對象包括：原始影

像、模型参数、中间特征和动态剪枝路径；最终推理结果仅按授权返回给请求方，不在服务端控制面中额外公开。系统允许暴露的对象仅包括最小必要的结构化元数据、审计摘要和最终分类结论。本作品不声称覆盖生产环境中所有攻击面，尤其不覆盖恶意共谋、模型抽取、物理侧信道和输出反推风险。系统的威胁模型、保护对象与安全边界核心定义如表 2-1 所示。表格基于两方半诚实威胁模型制定，仅覆盖原型系统设计范围内的安全假设与防护边界，不代表生产环境下的完整安全承诺，生产部署仍需补充身份认证、传输安全、密钥管理、长期审计和侧信道防护等机制。

表 2-2 系统各维度安全边界覆盖情况

边界项	当前覆盖	当前不覆盖或限制
输入明文保护	明文图像仅在浏览器工作线程内解码、裁剪与分片，服务端仅接收密态 share 与结构化元数据	不覆盖浏览器被完全控制或本地终端失陷后的明文窃取
模型参数保护	模型参数不向数据使用方以明文暴露，仅在两方安全执行域内参与计算	不覆盖恶意参与方串谋、模型抽取与生产级侧信道分析
动态决策隐私	PredictorLG、阈值判断与并列分数决策保留在安全执行域内	不覆盖进入安全图后的主动任务取消、长连接断开感知与更强攻击模型
协议与输入防护	原始多部件表单预检、JSON（JavaScript Object Notation）字节门、share 哈希与张量合法性快检、重放与并发守卫已落地	不覆盖生产级分布式 DoS、跨节点全局限流与外部 WAF/网关联动
审计与复现	nonce、请求载荷（payload）、质量摘要与审计事件落盘，报告与脚本可回溯	不覆盖长期合规归档平台、外部 SIEM 集成与跨系统追责链

各维度安全边界的具体覆盖情况与明确未覆盖范围如表 2-2 所示，上述边界说明用于避免将原型级控制面夸大为生产级防御体系。更完整的未覆盖范围、输出侧风险与原型部署边界，统一放在第 5.5 节“当前局限性”中说明。“当前覆盖”项中可由控制面和协议入口触发的边界已通过黑盒验证并留存审计证据；密码学安全性仍以两方

半诚实、不串谋威胁模型为前提。

为便于从步骤层面概括端到端链路，算法 1 给出本作品从终端预处理、秘密分享、服务端预检测到密态推理与结果回传的完整流程。该流程与图 2-2 所示时序一致，强调了明文仅停留在终端侧、跨域仅传输密态分片与审计摘要的边界约束。算法 1 中使用的主要符号说明如下： $\mathbf{x}$ 表示用户输入图像， $\mathcal{B}$ 表示离线冻结导出的部署 bundle， $\langle \theta \rangle$ 表示以秘密共享形式加载的模型参数份额， $\mathbf{y}$ 表示最终预测结果； θ_{cfg} 表示 bundle 中固化的模型与剪枝配置， τ^* 表示离线校准得到的最优判定阈值， $\mathcal{M}$ 表示 bundle 清单与元数据； $\langle \mathbf{x} \rangle$ 表示输入图像经本地预处理后生成的秘密共享分片， ℓ 表示当前 Transformer block 编号， L 表示模型总层数。

从部署视角看，威胁模型不仅决定密码学协议选择，也决定系统哪些组件能够接触明文、哪些组件只能处理密态分片。许多系统只在算法层声明“保护输入与模型”，但不进一步说明原始文件、预处理张量、结构化元数据和审计记录分别落在何处。本作品在威胁模型中显式规定：原始影像与明文像素止于终端侧，跨边界传输对象被限制为 share、结构化元数据和审计摘要，服务端仅在全部快检通过后触发安全执行。这种“边界先于协议”的约束，是后续控制面设计和部署骨架成立的前提。

算法 1. 端到端 2PC 推理流程

输入： 图像 x ，冻结部署 bundle $\mathcal{B}$ ，模型分片 $\langle \theta \rangle$

输出： 预测结果 y

- 1: 客户端对 x 进行预处理并准备本地分片
 - 2: 客户端提交元数据与安全载荷
 - 3: 服务端执行准入检查
 - 4: **if** 请求被拒绝 **then**
 - 5: **return** 拒绝码
 - 6: 加载 $\mathcal{B} = (\theta_{\text{cfg}}, \tau^*, \mathcal{M})$
 - 7: 使用 $\langle x \rangle$ 和 $\langle \theta \rangle$ 初始化安全执行状态
 - 8: **for** 模块 $\ell = 1, \dots, L$ **do**
 - 9: 执行安全线性层计算
 - 10: **if** $\ell \in \{3, 6, 9\}$ **then**
 - 11: 执行安全 token 选择
 - 12: 执行安全友好非线性与归一化
 - 13: 重构最终分数并施加 τ^*
 - 14: 返回 y 并追加审计记录
-

算法 1 端到端 2PC 推理流程

在安全边界之外，系统还需要处理“元数据最小化”问题。实际部署时，完全隐藏所有运行状态并不现实，因为服务端需要依据尺寸、哈希[38]、质量摘要和任务标

识完成预检、去重、审计与资源调度；但如果元数据过多，又可能间接泄露输入分布或动态路径。本作品采用最小必要原则，只保留验证任务完整性和运行可追溯所需的结构化字段，并将原始像素、明文张量、模型权重和剪枝决策限制在相应安全边界内。该设计为后续从本机展示样例迁移到医院侧与模型提供方分离部署提供了统一约束。

2.6 DynamicViT 密态改写与固定图部署语义

原始 DynamicViT 中四类核心操作的安全计算困难点、本作品的改写方案、代价与验证方式如表 2-3 所示。

为便于从工程层面理解本节中的协议友好重写，图 2-4 将原始 DynamicViT 的删除式剪枝路径与本作品的掩码化密态语义重写并列展示。其核心差异在于：正式交付链路不再直接删除词元，而是以 keep-mask 表达保留边界，进而在固定图安全执行约束下保留动态剪枝能力。

表 2-3 DynamicViT 原始操作与密态语义改写对照表

原始 DynamicViT 操作	安全计算困难	本作品改写方式	代价	验证方式
直接删除 token	变长结构难以进入固定图安全执行，且会暴露路径。	改写为保留/置零的掩码化表达，由安全选择完成保留决策。	保留了部分冗余计算。	本地明文参考路径与 secure replay 对齐。
明文阈值比较	数据相关分支会泄露剪枝边界。	将比较逻辑改写为安全比较语义。	增加比较通信与控制流复杂度。	部署阈值回代与输出一致性验证。
数据相关 Top-K 选择	动态索引与并列分数处理不稳定。	采用编码索引跟踪与排序网络构造保留掩码(keep-mask)。	排序代价上升。	排序结果与后续决策链功能验证。
外部明文决定 keep-mask	会退化为半隐私运行。	将 PredictorLG、阈值判断和并列分数决策保留在安全执行域内。	图规模扩大、执行时间增长。	双向隐私运行链与前端控制面联调。

从技术实现角度看，本作品并不没有采用简单把原始剪枝算子“搬进”安全执行

域，而考虑将原有的打分、比较、排序、删除与回排逻辑拆解为更适合固定图执行的两层语义：其一，由局部-全局词元预测器（local-global token predictor, PredictorLG）在两方安全执行环境内部产生词元重要性得分；其二，由编码键、排序网络与 keep-mask 更新逻辑在同形状张量上恢复保留边界。其中，PredictorLG 是 DynamicViT 中用于生成 token 重要性得分的轻量级预测模块，可根据当前层 token 表征估计各空间 token 对最终分类结果的贡献，并为后续剪枝提供保留分数。

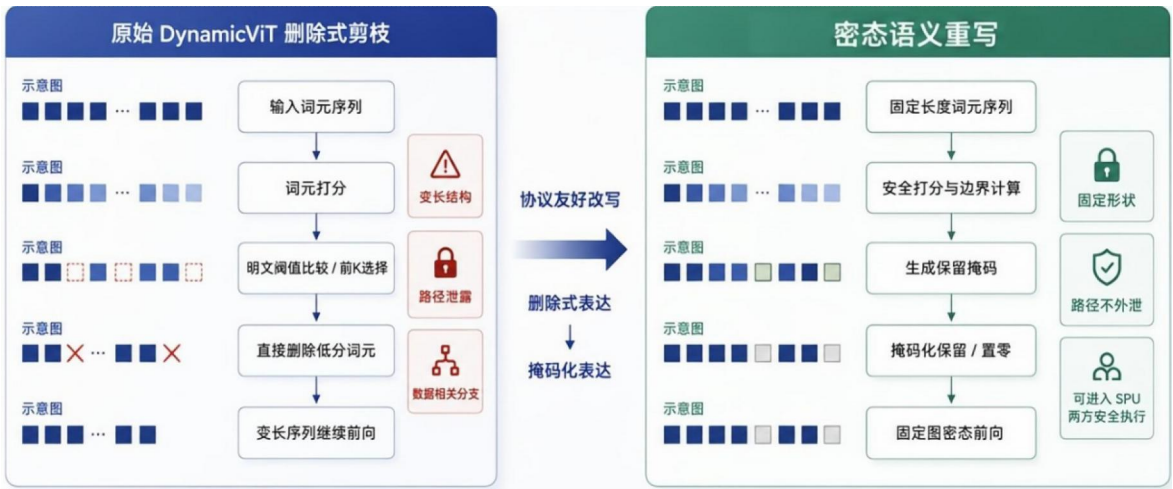


图 2-4 DynamicViT 密态改写前后对照图

这一重写能够稳定落地，依赖训练导出、冻结 bundle 与安全执行路径之间的一致性。训练侧保留第 3、6、9 个 stage 的 token 保留语义和 PredictorLG 判别边界；部署侧不重新构造启发式评分逻辑，而是读取同一组剪枝位置、保留数量、PredictorLG 参数与校准项，并在 SPU 中通过编码键、索引跟踪排序和 keep-mask 构造完成固定图安全选择。这样，原始 DynamicViT 的动态删除语义被转化为固定形状张量上的密态掩码更新，既避免外泄变长路径，又保留动态 token 边界的判别价值。

2.7 硬件部署与软件基准

算法、训练过程、离线精度、算子代理基准与鲁棒性证据均保持原稿不变；本轮仅使用原始代码在当前VPS上复跑32条医疗部署验证样本，环境基准如表

2-4所示。VPS复跑的性能与通信量来自同一次运行，通信量按环回接口TX增量单计；金融边界压力数据及其他历史实验记录仍保留原报告口径。两组数据硬件、SPU版本和通信计数方法不完全一致，因此后文按各自口径列示，不作直接比例比较；当前VPS的详细配置与医疗复跑数据以本轮实测记录为准，后续时延和通信量

均默认建立在本节明确标注的相应口径之上。

表 2-4 原型系统硬件部署与软件环境基准

项目	配置
CPU	阿里云KVM实例，Intel Xeon Platinum，16 vCPU (1路、8核、每核2线程)
物理内存	61 GiB (未配置Swap)
操作系统	Ubuntu 24.04.4 LTS
内核版本	Linux 6.8.0-136-generic
Python	Python 3.9.25
安全计算底座	SecretFlow SPU 0.9.3b0 (Apache License 2.0)
原型运行模式	JAX 0.4.30, NumPy 1.26.4, PyTorch 1.13.1+cpu; 两方2PC colocated localhost, batch size 8
GPU	未配置、未使用GPU; 本轮SPU推理使用CPU运行， GPU不参与性能统计。

2.8 展示样例与真实部署框架

为区分原型演示与后续部署边界，本作品在工程实现中将系统形态划分为“本机展示样例”和“真实部署框架”两类。本机展示样例用于验证端到端流程：系统在一台机器上启动本地双节点安全处理器单元（Secure Processing Unit, SPU），由展示站前端完成图像选择、预处理与分片生成，后端完成协议校验、审计记录和密态前向计算，形成可运行、可追踪的闭环。

真实两方部署迁移骨架面向医院侧与模型提供方分离部署，用于说明本机演示样例如何迁移到角色分离的 2PC 部署形态。该框架保留了 public manifest、P1/P2 party manifest、单方 share 加载、远程 2PC 配置生成、医院侧/AI 侧/协调侧三类网关角色以及部署包生成脚本。医院侧负责原始影像接收、预处理与 P1 share 留存，AI 或模型提供方负责 P2 share 与模型配置文件管理，协调侧负责公开任务状态、运行编排和最终可公开结果的返回。

对应到当前仓库保留的正式代码入口，这一“真实部署框架”并非停留在概念说明层面，而已经具备三类可核验骨架：第一类是 `split public manifest`、`P1/P2 party manifest` 与 `--party-local-share-load``，用于把医院侧与模型提供方的私有 share 输入边界在运行参数层面显式拆开；第二类是 `render-deployment`` 与 `start-party`` 一类部署脚本，用于把单机原型重写为医院侧节点、AI 侧节点和协调侧可分别启动的远程 2PC 配置；第三类是 `showcase_api.split_gateway`` 与 `tools/split_gateway_smoke.py`` 提供的最小三方网关框架，用于验证医院图片入口、AI 侧 P2 share 接收和协调方异步任务编排是否能够独立成立。。

为直观展示上述迁移骨架在物理与逻辑层面的信任边界，图 2-5 给出了当前原型向真实两方部署形态过渡时的部署拓扑。左侧为医院终端与本地明文处理域，中间为 WAN/DMZ 传输边界，右侧为业务前置网关、服务端权威快检层与参与方安全执行域。



图 2-5 系统物理与逻辑部署图

二者对应同一安全推理思想下的不同验证层次：本机端到端演示样例证明流程可运行、可审计、可复现；真实两方部署迁移骨架说明系统具备向医院侧与模型提供方分离部署的接口、配置和脚本基础。当前框架仍属于原型验证与部署迁移准备阶段，不直接覆盖生产环境中的 HIS/PACS 接入、机构身份认证、TLS/VPN、长期审计、任务队列、密钥管理和模型私有加载策略。

第三章系统实现

3.1 离线训练与部署准备实现

本作品的系统实现遵循“离线训练与部署准备、固定图安全化改写、在线 2PC 执行”三层主线。训练阶段保留 DynamicViT 的词元重要性建模与样本相关边界能力；部署阶段把明文删除式剪枝收束为固定图可执行的掩码化语义；在线阶段则由两方安全推理链路承接密态前向，并仅对外返回最终分类结果。

由此，训练阶段是整个部署链路的前置组成部分。当前实现仍以 DeiT-S 形状的 DynamicViT 变体为骨架，在第 3、6、9 个阶段执行词元筛选，并通过教师—学生蒸馏稳定动态裁剪后的表征，使训练期语义能够为后续安全部署提供可继承的边界信息。

离线阶段真正面向部署的交付物是由阈值搜索、冻结导出和 bundle 组织共同形成的部署资产。当前正式交付链路会将 `manifest.json`、`args\_snapshot.json`、`modified\_plaintext\_model\_state\_dict.pth`、`threshold\_best.json` 与导出命令记录共同固化，使在线 2PC 侧能够复用训练期确定下来的保留率、算子口径与判别阈值。

本作品在方法实现上关注的是训练语义、bundle 语义与在线执行语义的连续承接，不是让协议在脱离部署口径的条件下单独运行。若缺少这层冻结与导出，浏览器展示、服务端快检与 SPU 安全执行就无法保证引用同一套判别边界。

算法 2. base_rate 扫描与可部署保留率选择

输入： 候选 base-rate 集合 $\mathcal{R}$ ，warm-start 检查点 θ_0 ，验证集 $\mathcal{V}$ ，剪枝阶段 $\mathcal{L}_p = \{3, 6, 9\}$

输出： 选定的基础保留率 r^* 及 token 保留率 $(r^*, (r^*)^2, (r^*)^3)$

- 1: 初始化结果表 $\mathcal{M} \leftarrow \emptyset$
 - 2: **for** 每个 $r \in \mathcal{R}$ **do**
 - 3: 设置 token 保留率 $(r_3, r_6, r_9) \leftarrow (r, r^2, r^3)$
 - 4: 从 θ_0 初始化模型
 - 5: 在相同 warm-start 口径下训练或评估
 - 6: 计算验证指标：argmax 精度、阈值精度和 AUC
 - 7: 将 $(r, r_3, r_6, r_9, \text{Acc}_{\text{argmax}}, \text{Acc}_{\text{thr}}, \text{AUC})$ 追加到 $\mathcal{M}$
 - 8: 根据验证性能与部署代价选择 r^*
 - 9: 将 $(r^*, (r^*)^2, (r^*)^3)$ 冻结写入部署参数
 - 10: **return** $r^*, (r^*, (r^*)^2, (r^*)^3)$
-

算法 2 base_rate 扫描与可部署保留率选择

算法 3. 带蒸馏补偿的动态训练与部署对齐流程

输入： 训练集 $\mathcal{D}$, 教师模型 T , 学生 DynamicViT S_θ , 剪枝阶段 $\mathcal{L}_p = \{3, 6, 9\}$, 基础保留率 r , 蒸馏权重 $\lambda_{cls}, \lambda_{tok}$

输出： 面向部署对齐的检查点 θ^*

- 1: 设置 token 保留率 $(r_3, r_6, r_9) \leftarrow (r, r^2, r^3)$
- 2: 使用 $\mathcal{L}_p$ 与 (r_3, r_6, r_9) 配置 S_θ
- 3: **for** 每个小批次 $(x, y) \in \mathcal{D}$ **do**
- 4: $(y_s, a_s) \leftarrow S_\theta(x)$
- 5: $(y_t, a_t) \leftarrow T(x)$
- 6: $\mathcal{L}_{ce} \leftarrow \text{CE}(y_s, y)$
- 7: $\mathcal{L}_{cls} \leftarrow \text{KD}(y_s, y_t)$
- 8: $\mathcal{L}_{tok} \leftarrow \text{TokenKD}(a_s, a_t)$
- 9: $\mathcal{L} \leftarrow \mathcal{L}_{ce} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{tok}\mathcal{L}_{tok}$
- 10: 通过对 $\mathcal{L}$ 反向传播更新 θ
- 11: **if** 启用部署对齐模式 **then**
- 12: 在固定深度或 secure-static 设置下评估 S_θ
- 13: 记录 ‘secure_static_train_depth’ 与 ‘secure_static_skip_pruning’
- 14: 根据验证结果与部署对齐证据选择检查点 θ^*
- 15: **return** θ^*

算法 3 蒸馏补偿与部署对齐联动训练流程

算法 4. 阈值校准与冻结 bundle 导出流程

输入： 已校准模型检查点 θ^* , 选定 base rate r^* , 验证集 $\mathcal{V}$, 候选阈值集合 Θ

输出： 冻结部署 bundle $\mathcal{B}$

- 1: 在 $\mathcal{V}$ 上运行 S_{θ^*} 并收集正类概率 $\{p_i\}_{i=1}^n$
- 2: 初始化 $\theta_{best} \leftarrow 0.5, a_{best} \leftarrow 0$
- 3: **for** 每个阈值 $\theta \in \Theta$ **do**
- 4: **for** $i = 1$ 到 n **do**
- 5: $\hat{y}_i(\theta) \leftarrow \mathbb{I}(p_i \geq \theta)$
- 6: $a(\theta) \leftarrow \frac{1}{n} \sum_{i=1}^n \mathbb{I}(\hat{y}_i(\theta) = y_i)$
- 7: **if** $a(\theta) > a_{best}$ **then**
- 8: $\theta_{best} \leftarrow \theta$
- 9: $a_{best} \leftarrow a(\theta)$
- 10: 导出模型参数与剪枝配置
- 11: 写出包含 r^* 与部署参数的 ‘args_snapshot.json’
- 12: 写出包含 θ_{best} 与校准指标的 ‘threshold_best.json’
- 13: 写出描述 bundle 文件与加载规则的 ‘manifest.json’
- 14: 将全部文件打包为冻结部署 bundle $\mathcal{B}$
- 15: **return** $\mathcal{B}$

算法 4 阈值搜索、离线校准与冻结导出流程

为避免离线训练、部署选择和在线安全推理之间出现语义断裂，本作品将离线准备进一步整理为三个可复核算法：算法 2 描述 base_rate 扫描与可部署保留率选择流

程描述，算法 3 蒸馏补偿与部署对齐训练流程，算法 4 描述阈值搜索、离线校准与冻结导出流程。三者共同构成在线 2PC 推理之前的部署资产生成链路。

为便于阅读，算法 2 至算法 4 中使用的主要符号说明如下： $\mathcal{D}$ 表示训练集， $\mathcal{V}$ 表示验证集， T 表示教师模型， S_θ 表示参数为 θ 的学生 DynamicViT 模型； $\mathcal{L}_p = \{3,6,9\}$ 表示剪枝阶段集合， r 表示基础 token 保留率， (r, r^2, r^3) 表示三个剪枝阶段的目标保留率； λ_{cls} 与 λ_{tok} 分别表示分类蒸馏损失和 token 蒸馏损失的权重； $\mathcal{R}$ 表示候选 base_rate 集合， Θ 表示候选阈值集合； θ^* 表示经过训练与筛选后选定的模型参数， θ_{best} 表示离线阈值搜索得到的最佳分类阈值， $\mathcal{B}$ 表示最终冻结导出的部署 bundle。

3.1.1 ABY 风格混合表示与协议适配

标准 ViT 的安全推理实现，通常需要同时面对两类代价：一类来自线性层、矩阵乘法和加法的高维张量计算，另一类来自比较、符号判断、非线性激活与归一化中的跨域操作。在秘密共享环境下，这两类操作并不适合统一在同一种共享表示中完成。为此，本作品在系统实现中借鉴了 ABY [10] 类混合协议的核心思想，即根据操作类型在算术共享与布尔共享之间进行按需切换，从而避免将全部计算强行压缩到单一表示下执行。

需要说明的是，本作品并未直接采用 ABY2.0 作为完整推理主协议，也未重新实现一套独立的 ABY 推理框架。本作品的正式部署链仍以现有秘密共享推理框架为主体，而 ABY 在本作品中的作用主要体现为一种“混合表示使用原则”：线性层、残差加法、张量乘法等数值计算尽量保留在算术共享域中执行；比较、最高位提取、阈值判断及部分非线性相关操作则在必要时转入布尔共享域处理，完成后再返回算术域继续后续流水。因而，更准确的表述应为：本作品吸收了 ABY 风格的混合表示思想，并将其局部嵌入到安全推理链中，而不是将整套系统写作“基于 ABY 协议实现”。

设算术共享张量为 $[x]_A$ ，布尔共享张量为 $[x]_B$ 。本作品在底层主要使用两类转换原语：A2B 与 B2A。其基本思路是，当某一步骤需要执行比较或符号位相关操作时，先将 $[x]_A$ 转换为 $[x]_B$ ，在布尔域中完成最高位提取、大小判断或布尔组合，再将结果通过 B2A 转回算术域，以便参与后续乘法、加法或张量融合。该过程可概括为

$$[x]_A \xrightarrow{A2B} [x]_B \xrightarrow{\text{Compare/Logic/MSB}} [b]_B \xrightarrow{B2A} [b]_A$$

上述转换是为了将真正依赖比较的部分局部剥离出来，从而避免在算术域中直接模拟复杂比较逻辑所导致的高额开销。因此，本作品的方法重点是尽量减少不必要的跨域切换次数，把共享表示转换限定在真正不可避免的子过程上。前文实验已经表明，在当前实现中，Beaver triple、Binary triple 与随机预处理材料本身并非通信主瓶颈，而 A2B/B2A 这类共享表示转换对网络条件更为敏感。因此，本作品后续的函数改造策略，实质上都围绕同一个目标展开：尽量减少需要通过比较、符号判定和复杂非线性才能完成的计算环节，从而间接减少 A2B/B2A 触发频率及其带来的在线通信代价。

从系统角度看，这种 ABY 风格的局部适配还带来两点好处。第一，它保留了线性层在算术共享下的高吞吐特性，使大规模矩阵运算不必迁移到更不擅长数值计算的共享形式中；第二，它为非线性、阈值判断与裁剪逻辑提供了更清晰的实现边界，使得本作品能够在不改变整体推理框架的前提下，对单个高代价算子进行定向重写。因此，本作品将混合表示视为一种底层实现策略，而真正决定系统性能上限的，则是上层算子本身是否被改造成了适合秘密共享环境的计算形式。

基于上述混合表示与局部转换机制，本作品进一步对原始 ViT 中最不利于安全推理的激活函数、注意力归一化与归一化层进行了针对性改造。

3.1.2 面向安全推理的函数改造策略

A. 激活函数改造：由 GELU 向平方型激活迁移

在标准 ViT 中，多层感知机（MLP）部分通常采用 GELU 作为激活函数，其形式为

$$\text{GELU}(x) = \frac{1}{2}x \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right).$$

GELU 在明文环境中具有良好的平滑性和表达能力，但在秘密共享环境中通常需要借助误差函数、指数函数或高阶近似才能实现，因此会显著增加比较、截断、乘法深度和交互轮次。就正式部署而言，GELU 是原始 ViT 安全化过程中最敏感的非线性之一。

针对这一问题，本作品采用“低阶多项式替代复杂平滑非线性”的改写思路。正式部署主线使用 `fixed_square`，将激活计算压缩为乘法与常数缩放；训练与诊断阶段则保留可学习二次激活等支路，用于观察表达能力与安全友好性之间的折中。

$$\phi_{\text{fixed}}(x) = \alpha x^2,$$

$$\phi_{\text{learn_sq}}(x) = \alpha x^2,$$

$$\phi_{\text{quad}}(x) = \alpha x^2 + \beta x.$$

上述候选形式中，`fixed_square` 对应正式部署主线，可学习二次激活主要用于训练与代理分析。换言之，本作品不会把所有平方型候选都写成已正式部署的函数族，而是明确区分“最终 `bundle` 采用何者”与“训练期或代理实验曾比较何者”。因此，激活路径的正式口径是以可部署性为目标，在训练阶段保留必要灵活性，在导出阶段收束到 `fixed_square` 主线，以获得更稳定、更低交互成本的安全执行路径。

激活路径	数学形式	用途
<code>fixed_square</code>	αx^2	正式部署主线
<code>learnable_square</code>	αx^2	可学习系数的训练候选
<code>learnable_quadratic</code>	$\alpha x^2 + \beta x.$	提升近似能力的训练/分析路径
<code>learnable_quadratic_gelu_init</code>	$\alpha x^2 + \beta x.$	以 GELU 拟合为初始化的训练路径

这种区分对于复现同样重要。若不把训练支路、代理支路与正式部署支路分开描述，读者容易误以为所有候选函数都已进入最终 `bundle`，从而混淆实验结论与交付口径。

此外，系统还在部分路径中对激活输入施加 `clip` 控制，以抑制定点域中的数值扩张风险。平方运算会放大输入动态范围，因此 `clip` 是与有限域、定点精度和后续归一化协同设计的一部分。

B. 注意力改造：由标准 Softmax 向低交互注意力迁移

标准自注意力中的归一化过程通常写作

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V.$$

在标准自注意力中，`Softmax` 需要同时计算指数、求和与除法，是秘密共享环境下典型的高代价算子。当注意力矩阵维度增大时，其在线通信量与执行时延会迅速上升，因此成为影响 ViT 安全推理可用性的另一关键瓶颈。

为降低这一部分的代价，本作品将注意力改造明确划分为正式部署路径与代理对比路径。正式部署主线采用 `uniform attention`，即不再显式构造标准 `Softmax` 权重，而是直接使用均匀分配形式完成 `value` 聚合。

$$A_{\text{uniform}} = \frac{1}{n} \mathbf{1}\mathbf{1}^\top,$$

这种写法弱于标准 Softmax 的精细概率表达，但它显著减少了指数的、比较与归一化开销，因此被纳入当前正式 bundle。对于本作品，uniform attention 的意义在于提供可稳定部署的低交互主线。

在此基础上，本作品仍保留若干近似 Softmax 的代理路径，用于分析不同归一化替代对通信量与时延的影响。这些路径的共同目的是帮助判断注意力归一化本身是否构成主要瓶颈。

因此，正文需要明确区分“正式部署采用 uniform attention”与“代理实验曾比较其他近似路径”这两类结论。前者属于最终交付口径，后者只服务于代价分解与方法解释。

注意力路径	说明	角色
标准 Softmax	指数归一化	原始 ViT 基线
uniform attention	均匀权重聚合	正式部署主线
<i>softmax_2RELU</i>	低复杂度近似 Softmax	代理对比路径
<i>softmax_2QUAD</i>	二次近似 Softmax	代理对比路径

C. 归一化改造：由标准 LayerNorm 向可部署校准路径迁移

除了激活与注意力之外，归一化层也是安全推理中不可忽视的成本来源。标准 LayerNorm 可表示为

$$\text{LN}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta,$$

其中，均值与方差计算会引入平方、求和以及倒数平方根等高代价步骤，因此标准 LayerNorm 在秘密共享环境中同时面临乘法深度和数值稳定性压力。

针对这一问题，本作品并没有直接删除归一化层，而通过采用“保留归一化功能、压缩秘密域负担”的改写思路。正式部署主线采用 public-calibrated LayerNorm，通过离线校准或公开统计量约束运行时归一化；exact LayerNorm 则保留为对照或分析路径，用于维持与原始模型更接近的数值语义。

因此，LayerNorm 路径同样需要区分正式主线与代理/对照支路。本作品真正交付的是 public-calibrated LayerNorm，而不是把所有归一化路径都写成正式部署结果。

D. 改造策略的整体设计原则

综合来看，本作品对安全推理函数的改造是围绕秘密共享环境下的真实代价结构进行整体重构。

第一，尽量将大规模线性运算保留在算术共享域中执行，以避免不必要协议迁移。

第二，尽量用 low-degree polynomial、uniform aggregation 或 calibrated normalization 替代高代价非线性，其中正式部署主线分别对应 fixed_square、uniform attention 和 public-calibrated LayerNorm。

第三，尽量减少 A2B/B2A 等跨域转换的触发次数，使函数改造服务于协议友好性，而不是只追求局部函数形式更简单。

基于上述原则，本作品最终形成了以 fixed_square、uniform attention 和 public-calibrated LayerNorm 为核心的正式部署算子族，并将可学习二次激活、近似注意力与 exact LayerNorm 等路径保留为训练、代理分析或对照分支。

3.1.3 固定图安全化重写

原始 DynamicViT 的动态剪枝过程通常表现为“打分—排序—删除—重排”的变长执行路径。该路径在明文环境下可以直接减少后续 token 数量，但在两方安全推理中会带来两个问题：第一，删除后的 token 数量和位置与输入样本相关，若作为外部控制流暴露，可能泄露输入分布或中间特征；第二，变长张量会破坏安全计算后端对固定计算图、固定张量形状和稳定编译路径的要求。故本作品是将其改写为固定图下的 keep - mask 更新问题。

设第 l 个剪枝阶段输入空间 token 表示为 $X_l = \{x_{l,1}, x_{l,2}, \dots, x_{l,N_l}\}$ ，其中 N_l 为该阶段进入剪枝模块的空间 token 数。令上一阶段传入的保留掩码为 $m_l \in \{0,1\}^{N_l}$ ，其中 $m_{l,i} = 1$ 表示第 i 个 token 当前仍处于活跃状态， $m_{l,i} = 0$ 表示该 token 已在前序阶段被抑制。PredictorLG 在安全执行域内对活跃 token 产生重要性得分 $s_l = f_{\theta}^{\text{pred}}(X_l, m_l)$ ，其中 $s_{l,i}$ 表示第 i 个 token 在当前阶段的保留价值。为了避免直接暴露 token 重要性排序，本作品将排序与选择过程限制在两方安全执行域内部完成。为保证并列分数情况下的确定性，本作品引入编码键： $k_{l,i} = s_{l,i} - \varepsilon i$ ，其中 ε 为极小正常数。该编码方式使得当两个 token 得分相同时，较小索引具有稳定优先级，从而避免不同运行环境下出现不一致的 $\text{top} - K$ 选择结果。对已经失活的 token，本作品将其

编码键置为负无穷：

$$\tilde{k}_{l,i} = \begin{cases} k_{l,i}, & m_{l,i} = 1, \\ -\infty, & m_{l,i} = 0. \end{cases}$$

随后，将 $\tilde{k}_l$ 补齐到长度 $N_l^{\text{pad}} = 2^{\lceil \log_2 N_l \rceil}$ ，并在安全执行域内调用带索引跟踪的 bitonic sorting network 进行降序排序。排序后直接取前 K_l 个索引构造新的保留掩码：

$$m_{l+1,i} = \begin{cases} 1, & i \in \text{TopK}(\tilde{k}_l, K_l), \\ 0, & \text{otherwise.} \end{cases}$$

其中 $K_l = \lceil r_l N_l \rceil$ ， r_l 为该阶段的目标保留率。对于正式部署主线，本作品采用由离线 base_rate 扫描确定的三阶段保留率： $(r_3, r_6, r_9) = (r, r^2, r^3)$ 。

这一改写的关键在于：模型前向仍保持固定张量形状，token 是否继续参与后续计算由密态 keep-mask 控制，而不是通过外部可见的变长删除操作实现。也就是说，动态剪枝的判别语义被保留，但其执行形式从“明文变长路径”转换为“固定图掩码更新”。

算法 5. 固定图约束下的安全动态 Token 剪枝

输入： Token 分数 $s_l \in \mathbb{R}^{N_l}$ ，上一层保留掩码 $m_l \in \{0, 1\}^{N_l}$ ，保留预算 K_l

输出： 更新后的保留掩码 $m_{l+1} \in \{0, 1\}^{N_l}$

```

1: 设置  $\varepsilon \leftarrow 10^{-6}$ 
2: for  $i = 1$  到  $N_l$  do
3:    $k_i \leftarrow s_{l,i} - \varepsilon i$ 
4:   if  $m_{l,i} = 0$  then
5:      $\tilde{k}_i \leftarrow -\infty$ 
6:   else
7:      $\tilde{k}_i \leftarrow k_i$ 
8:  $N_l^{\text{pad}} \leftarrow 2^{\lceil \log_2 N_l \rceil}$ 
9: 将  $\tilde{k}$  与索引向量  $I = (1, \dots, N_l)$  填充到长度  $N_l^{\text{pad}}$ 
10:  $(\tilde{k}^\downarrow, I^\downarrow) \leftarrow \text{BitonicSortDesc}(\tilde{k}, I)$ 
11:  $S_l \leftarrow \{I_1^\downarrow, \dots, I_{K_l}^\downarrow\}$ 
12: for  $i = 1$  到  $N_l$  do
13:    $m_{l+1,i} \leftarrow \mathbb{I}(i \in S_l)$ 
14: return  $m_{l+1}$ 
    
```

算法 5 固定图约束下的安全动态 Token 剪枝

上述算法对应服务器实现中的 ``_secure_build_keep_decision`` 与 ``_bitonic_sort_desc_with_indices`` 路径。具体而言，系统先根据 token 得分和索引构造 encoded-key，再将无效 token 屏蔽为负无穷，随后补齐到 2 的幂次并执行带索引跟踪的 bitonic 降序排序，最后直接利用前 K_l 个索引构造 keep-mask。该路径不

再采用“恢复阈值后再对全部 token 执行一次比较”的两阶段写法，因此减少了一次额外的全量比较扫描。

从安全边界看，该改写满足两个性质。第一，外部执行图的张量形状不随输入变化，服务端控制面只能观察到固定形状的密态分片和固定阶段的执行过程，不能直接获得某个样本被删除的 token 位置。第二，排序、top-K 选择与 keep-mask 构造均发生在安全执行域内部，动态剪枝决策不会以明文控制流形式暴露给任一单方。

从复杂度看，bitonic sorting network 在补齐长度 N_l^{pad} 上的深度为 $D(N_l^{\text{pad}}) = O(\log^2 N_l^{\text{pad}})$ ，比较器数量为 $C(N_l^{\text{pad}}) = O(N_l^{\text{pad}} \log^2 N_l^{\text{pad}})$ 。若单次安全比较交换的平均通信代价记为 (c_{cs}) ，则单阶段剪枝边界求解的通信量可表示为

$$T_{\text{comm}}(N_l^{\text{pad}})O(N_l^{\text{pad}} \log^2 N_l^{\text{pad}}) \cdot c_{\text{cs}}.$$

由于前序阶段的 keep-mask 会减少后续活跃 token 数量，实际有效比较规模会随剪枝阶段推进而下降。因此，动态剪枝的价值不仅体现在保留判别边界，也体现在后续安全比较、排序和掩码传播代价的逐阶段收缩上。

3.1.4 执行结构与边界约束

图 3-1 从实现结构角度补充展示当前项目框架：上层对应医院终端、业务网关与双节点密态执行域；中层分别对应控制面闭环、DynamicViT 安全化重写组件与两方半诚实安全计算协议；底层则是秘密分享、比较/选择与审计摘要等密码学基础原语。该图用于说明第 2 章“系统设计”与第 3 章“系统实现”的衔接关系。

图 3-1 可按“终端本地明文处理—跨域密态传输—服务端入口门控—双节点密态执行—结果回传与审计固化”五个层次理解。图左侧的医院终端代表数据提供方，也是整条链路中唯一允许接触原始影像、明文像素张量和局部质量指标的位置；图右侧的双节点密态执行域代表模型持有方与协同计算方在两方半诚实安全计算约束下共同完成推理的区域；两者之间由业务网关和服务端控制面构成一道明确的信任边界。

从输入侧开始，用户在浏览器端选择影像后，主线程并不直接承担高开销计算，而只负责文件句柄保留、界面状态维护与请求编排。真正的图像头部尺寸检查、异常尺寸阻断、解码、中心裁剪、像素归一化、本地数据质量评估、审计哈希链构建以及 share0/share1 的小端序 Float32 序列化，全部在浏览器工作线程内部完成。图 3-1 中“医院终端”内部已经同时完成了明文处理、质量预检与秘密分享三项职责，这是

原始像素不越出终端侧的关键。

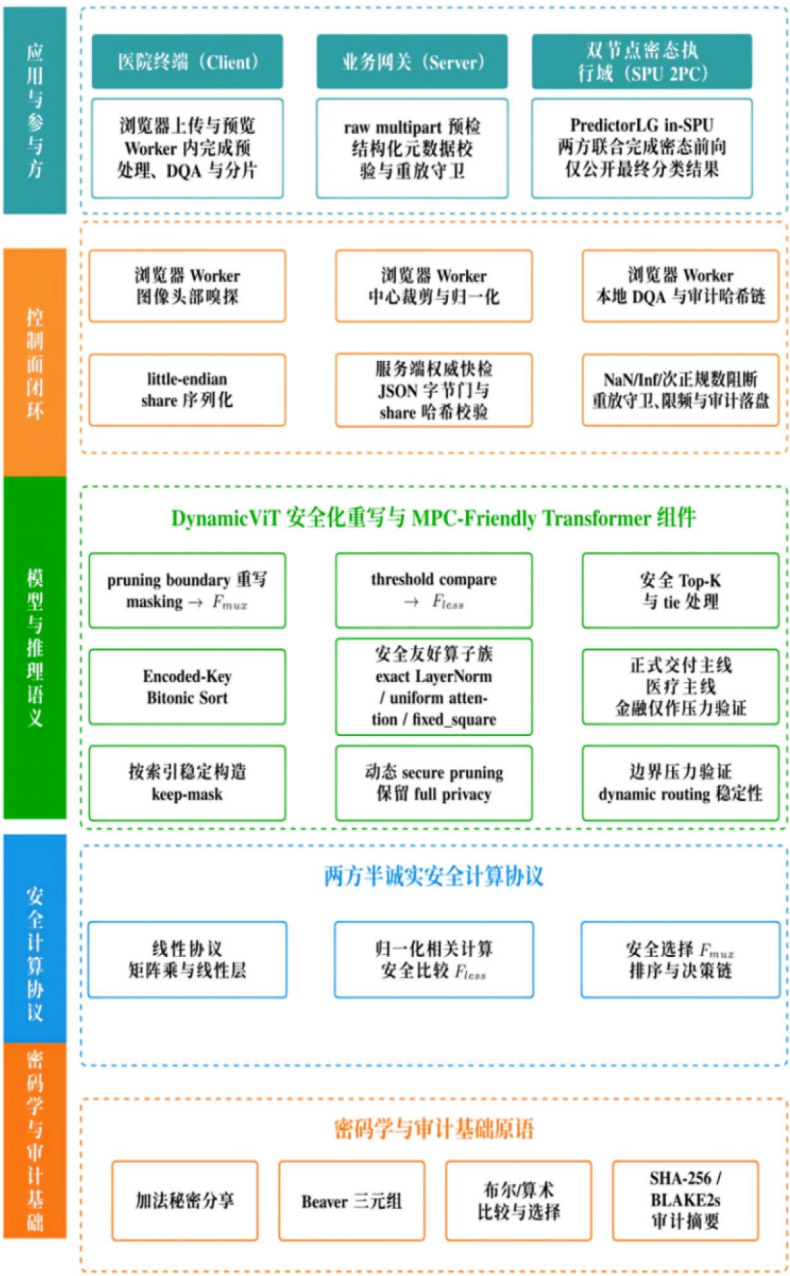


图 3-1 密捷项目当前框架总体架构图。

当终端侧形成两份密态分片与结构化元数据后，跨越信任边界传输的对象就被严格限制为：share 字节流、审计摘要和最小必要的结构化质量信息，而不是原始图像或可逆像素值。进入服务端后，业务网关首先负责请求体大小、boundary 参数、多部件结构和字段集合的快速预检；随后，权威快检模块继续完成 JSON 字节长度门、share 哈希校验、内存对齐复制、非有限数与次正规数阻断、张量幅值检查、质量摘要容差

复核、nonce 重放拦截、payload 去重以及来源地址并发守卫。只有在这些门控全部通过后，请求才会被转送至下游 SPU 密态执行域。

图 3-1 可按“终端本地明文处理—跨域密态传输—服务端入口门控—双节点密态执行—结果回传与审计固化”五个层次理解。医院终端是整条链路中唯一允许接触原始影像、明文像素张量和局部质量指标的位置；跨越终端边界后，系统仅传输密态分片、结构化元数据和审计摘要。业务网关与服务端控制面构成进入安全执行域之前的边界层，负责完成请求结构、张量合法性、哈希关系、重放状态和并发状态检查。

中部的 DynamicViT 安全化重写组件说明，本作品对动态词元剪枝路径本身进行协议友好改写。PredictorLG 在 SPU 内部生成词元得分，编码键排序网络在固定图中推导保留边界并构造 keep-mask，后续 attention、MLP、残差连接和分类头始终保留在两方安全执行域内运行。这样既保留了按样本变化的动态剪枝能力，又避免把剪枝决策暴露为外部明文控制流。在输出侧，SPU 域仅释放最终分类结论，不公开中间特征、模型参数或任一方明文 share。服务端控制面同步返回质量裁决、审计摘要和控制面耗时，并将关键事件写入审计记录。

因此，图 3-1 不仅展示系统模块组成，更说明了各类数据在何处生成、跨越哪些边界、由谁执行校验以及最终以何种最小暴露形式返回结果。

3.2 在线两方安全推理流程与模块协同

结合图 3-1 的当前项目框架，当前端到端链路可以压缩为三个安全阶段。第一阶段是终端本地预处理：浏览器工作线程完成影像解码、质量评估、审计链构建与秘密分享，安全目标是使原始影像和明文像素止于终端侧。第二阶段是服务端边界拦截：前置网关与权威快检模块在进入 SPU 之前完成报文、结构化元数据、share 字节流与重构张量合法性校验，安全目标是使异常输入、重放请求和畸形张量止于密态执行入口之外。第三阶段是 SPU 密态执行：PredictorLG、编码键排序、keep-mask 构造与前向分类全部保留在两方安全执行域内，安全目标是避免动态决策链退化为外部明文回放。

本节强调各阶段的安全职责分工：前端负责明文边界与分片生成，服务端负责权威阻断与审计固化，SPU 路径负责完成双向隐私约束下的动态安全推理。

3.3 前端控制面实现

前端实现采用主线程与单实例浏览器工作线程分离的结构。主线程只负责文件句

柄保留、页面状态维护和请求编排；工作线程内部完成图像头部尺寸嗅探、异常尺寸阻断、解码与中心裁剪、数据质量指标提取、源图与 share 审计哈希计算、小端序 32 位浮点（Float32）字节流序列化与可转移数组缓冲区传递。该设计避免了大体量数组在主线程上的结构化克隆开销，同时将高开销计算与页面交互解耦。

从浏览器执行路径看，前端把“图像解码—归一化—质量评估—share 序列化—审计哈希绑定”组织成一条本地完成的短链。仓内附录代码节选已经给出 little-endian Float32 share 写入实现；相应地，前端控制面在工程上保证了三件事：一是原始像素不需要以明文张量形式跨越浏览器边界；二是主线程只承担交互与调度，不因大数组复制而卡死；三是服务端收到的对象已经从原始图像转换为更适合进入协议校验和安全执行域的结构化 share 载荷。

在工程细节上，前端控制面的关键把每一步明文处理都收束在可审计的本地边界内。浏览器工作线程完成解码后，会立即执行尺寸门、中心裁剪、归一化、质量指标提取与哈希绑定，随后才生成两份 additively split share，并以 little-endian Float32 方式写入可转移缓冲区。采用 transferable ArrayBuffer 而非主线程复制，一方面减少了页面线程的阻塞和大张量的重复拷贝，另一方面也使“原始像素止于终端侧、跨域仅传 share”这一边界约束在实现层面具备可核验的物理落点。

此外，前端发往服务端的对象并不只有两份 share 字节流。当前控制面在请求中还同时携带 request manifest、客户端质量摘要、审计载荷以及控制面度量信息，使服务端能够在不恢复原始像素的前提下复核输入形状、哈希关系、质量阈值与请求来源状态。换言之，前端控制面的职责不仅是分片生成，更是为后续服务端权威快检准备一套最小必要而又可核验的结构化上下文。

前端计算过程中的关键数值变换、两方分片关系与审计关系见式(3-1)至式(3-3)，其中式(3-2a)进一步给出输入张量与两方 share 之间的对应关系：

$$x_{c,h,w} = \text{clip}\left(\frac{p_{c,h,w}}{\sigma_c} \mu_c, -2, 2\right) \quad (3-1)$$

$$\text{share0}_{c,h,w} = r_{c,h,w}, \quad r_{c,h,w} \sim U[-2, 2], \quad \text{share1}_{c,h,w} = x_{c,h,w} - \text{share0}_{c,h,w} \quad (3-2)$$

$$x_{c,h,w} = \langle x_{c,h,w} \rangle_0 + \langle x_{c,h,w} \rangle_1, \quad \langle x_{c,h,w} \rangle_0 = \text{share0}_{c,h,w}, \quad \langle x_{c,h,w} \rangle_1 = \text{share1}_{c,h,w} \quad (3-2a)$$

$$H_{\text{audit}} = \text{SHA256}(\text{v7} \parallel \text{nonce} \parallel H(\text{src}) \parallel H(x) \parallel H(\text{share0}) \parallel H(\text{share1})) \quad (3-3)$$

其中， $p_{c,h,w}$ 表示裁剪后图像在通道 c 、位置 (h,w) 的原始像素值， μ_c 与 σ_c 分别对应 ImageNet[36]均值与标准差， $x_{c,h,w}$ 为送入模型的归一化张量值；式（3-2a）中的 $\langle x \rangle_0$ 与 $\langle x \rangle_1$ 对应浏览器端生成的两份 additively split share，并在传输层分别落到 share0 与 share1。为避免浏览器端异常数值写入底层缓冲区，前端会先将归一化结果裁剪到 $[-2,2]$ ，再以 little-endian Float32 方式写入 share 字节流；审计链则把源图哈希、归一化张量哈希与两份 share 哈希绑定到同一个 nonce 上。

为便于直观看清主线程与浏览器工作线程的职责边界，图 3-2 将文件句柄保留、页面状态维护、可转移数组缓冲区传递、图像预处理、审计哈希与秘密分享生成拆解为三个协同区域。该图对应本节实现要点：主线程只负责交互与调度，工作线程承担高开销计算，并在跨越信任边界前完成明文终止与密态分片生成。

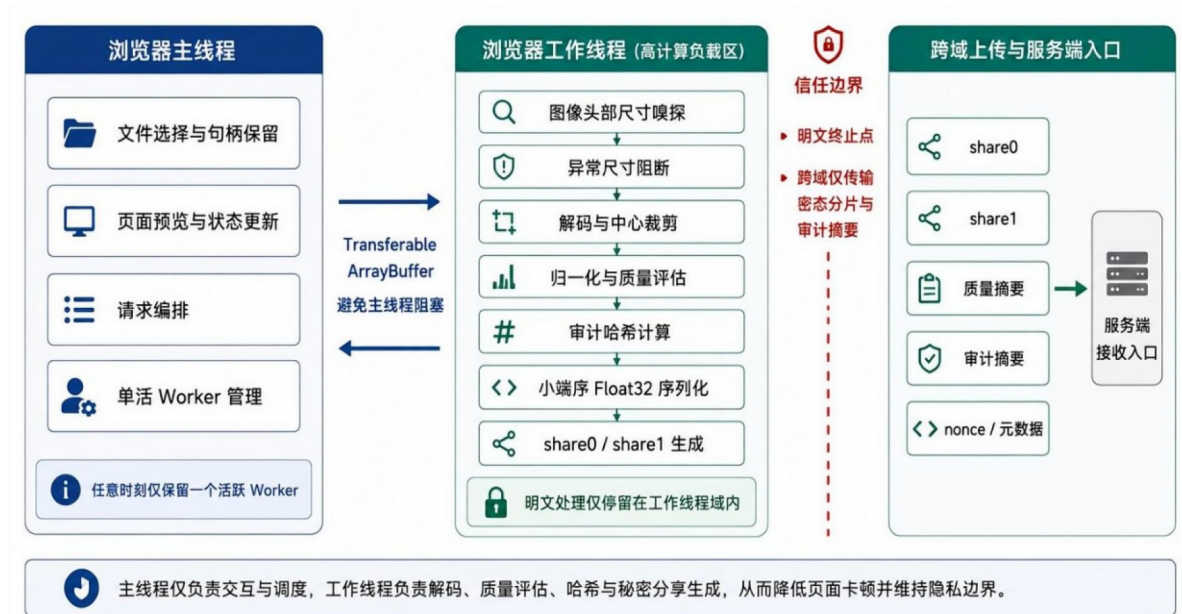


图 3-2 浏览器主线程与工作线程协同图

3.4 服务端权威快检与审计

在线密态推理链路中，服务端并不接触用户原始明文图像，但仍必须承担请求准入、结构校验、重放防护、并发门控和审计留痕等职责。若缺少这一层控制面，任意畸形报文、重复请求或资源异常都可能被直接送入 SPU 安全执行域，导致协议失败、资源占用异常或审计链断裂。

设一次在线推理请求为

$$q = (h, p, z, \tau),$$

其中 h 表示请求头与结构化元数据， p 表示上传的密态分片或密文载荷， z 表示由客户端生成的质量摘要、哈希摘要和审计字段， τ 表示请求时间戳与 `nonce`。服务端准入函数记为

$$\mathcal{A}(q) \in \{\text{accept}, \text{reject}(c)\},$$

其中 c 为拒绝原因。只有当 $\mathcal{A}(q) = \text{accept}$ 时，请求才被允许进入后续 SPU 密态推理链路。

本作品将准入函数拆分为五类检查：

$$\mathcal{A}(q) = \mathcal{G}_{\text{busy}} \circ \mathcal{G}_{\text{replay}} \circ \mathcal{G}_{\text{payload}} \circ \mathcal{G}_{\text{numeric}} \circ \mathcal{G}_{\text{shape}}(q).$$

其中， $\mathcal{G}_{\text{shape}}$ 检查报文结构、字段完整性、文件数量、张量维度和边界参数； $\mathcal{G}_{\text{numeric}}$ 检查数值范围、NaN/Inf、异常尺寸与非法标量； $\mathcal{G}_{\text{payload}}$ 根据载荷摘要识别重复或冲突请求； $\mathcal{G}_{\text{replay}}$ 根据 `nonce` 和时间戳执行重放防护； $\mathcal{G}_{\text{busy}}$ 根据当前 `inflight` 状态执行并发门控。若任一检查失败，服务端立即返回拒绝状态，并写入审计日志。

在实现层面，服务端返回的典型拒绝状态包括：

$$c \in \{\text{duplicate}_{\text{nonce}}, \text{duplicate}_{\text{payload}}, \text{busy}_{\text{retry_later}}, \text{invalid_shape}, \text{invalid_numeric}, \text{malformed_request}\}.$$

其中，`duplicate_nonce` 表示请求 `nonce` 已被使用，存在重放风险；`duplicate_payload` 表示载荷摘要与历史请求重复或冲突；`busy_retry_later` 表示当前安全执行域已有任务占用，服务端拒绝并发进入 SPU。上述状态是服务端正式准入语义的一部分。算法 6 在进入 SPU 前完成结构、数值、摘要、重放与并发门控。

为便于对应服务端控制面的实际落地点，图 3-3 将权威快检与审计闭环拆解为原始请求体读取、协议结构预检、JSON 长度门、`share` 哈希校验、张量快检、重放并发守卫与审计落盘等连续门控步骤。请求仅在全部门控通过后进入 SPU，任一层失败均立即阻断并记录审计结果。

该图与本节公式部分共同说明：客户端声明不会直接成为最终裁决依据，请求只有在全部分检查通过后才进入 SPU 密态执行域。

算法 6. 服务端准入与审计协议

输入： 请求 $q = (h, p, z, \tau)$ ，重放缓存 $\mathcal{R}$ ，载荷缓存 $\mathcal{P}$ ，在途标志 b

输出： 准入结果 $y \in \{\text{accept}, \text{reject}(c)\}$

```

1:  $e \leftarrow \text{InitAuditEntry}(q)$ 
2: if  $\neg \text{ValidShape}(h, p)$  then
3:    $c \leftarrow \text{invalid\_shape}$ 
4:   return  $\text{RejectAndLog}(e, c)$ 
5: if  $\neg \text{ValidNumeric}(p, z)$  then
6:    $c \leftarrow \text{invalid\_numeric}$ 
7:   return  $\text{RejectAndLog}(e, c)$ 
8:  $\rho \leftarrow \text{PayloadFingerprint}(p, z)$ 
9: if  $\rho \in \mathcal{P}$  then
10:   $c \leftarrow \text{duplicate\_payload}$ 
11:  return  $\text{RejectAndLog}(e, c)$ 
12: if  $\tau.\text{nonce} \in \mathcal{R}$  then
13:   $c \leftarrow \text{duplicate\_nonce}$ 
14:  return  $\text{RejectAndLog}(e, c)$ 
15: if  $b = \text{true}$  then
16:   $c \leftarrow \text{busy\_retry\_later}$ 
17:  return  $\text{RejectAndLog}(e, c)$ 
18:  $\mathcal{R} \leftarrow \mathcal{R} \cup \{\tau.\text{nonce}\}$ 
19:  $\mathcal{P} \leftarrow \mathcal{P} \cup \{\rho\}$ 
20:  $b \leftarrow \text{true}$ 
21:  $e.\text{status} \leftarrow \text{accepted}$ 
22:  $\text{AppendAuditLog}(e)$ 
23: return  $\text{accept}$ 
    
```

算法 6 服务器侧统一准入与审计协议

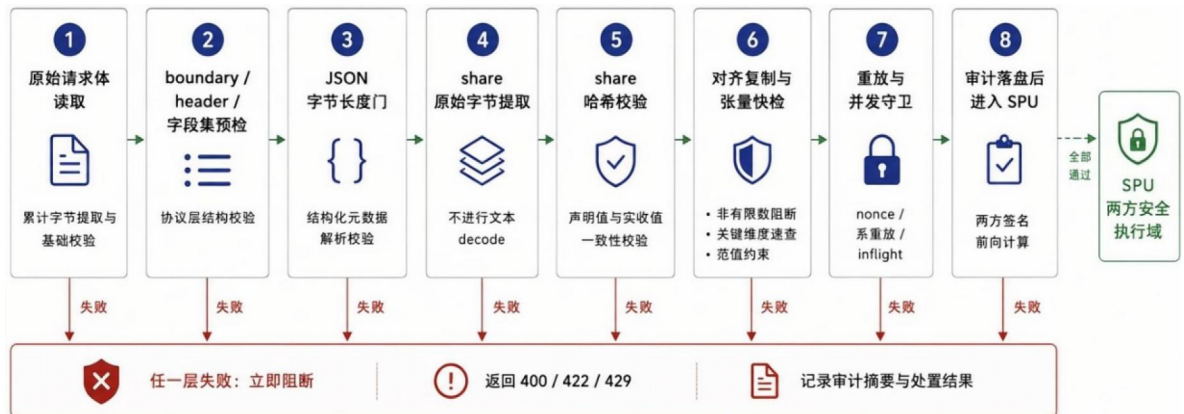


图 3-3 服务端权威快检与审计闭环图

该模块的价值主要体现在“边界控制”和“证据留存”两方面。前者保证服务端

只把结构完整、数值合法、未重放且未冲突的请求交给 SPU；后者保证每次接收、拒绝、执行和返回都有可复核记录。结合前端本地分片与审计哈希链，服务端权威快检使在线推理链路形成从客户端输入、密态传输、准入控制到安全执行的闭环。

第四章测试方案与结果分析

4.1 测试环境、数据与评价指标

本次测试使用的数据集规模、用途与核心评价指标如表 4-1 所示。实验主要使用 524 张验证集样本进行医疗主线评估，另使用 8 条金融边界压力样本进行迁移性与稳定性验证。医疗主线采用阈值精度、AUC（Area Under the ROC Curve）[42]和 `argmax` 精度作为主要指标，其中阈值精度用于反映部署口径下的最终分类效果，AUC 用于衡量整体判别能力，`argmax` 精度用于描述未校准条件下的直接决策表现。协议层与控制面验证则采用异常输入拦截率、重放拦截率、并发守卫结果和审计落盘状态作为主要观察指标。

需要说明的是，本作品中涉及的性能结果均以固定检查点和同口径验证集为基础，反映的是部署态结果而非多随机种子的训练均值。

表 4-1 测试数据集与评价指标说明

对象	样本规模	作用	主要指标
医疗全量验证集	524 张	正式阈值搜索与精度评估	<code>argmax</code> 精度、阈值精度、部署阈值
医疗部署验证批次	32 张	完整双向隐私原型运行	平均时延、双向总通信量、隐私边界
金融边界压力样本	8 条	极端分布输入下的稳定性验证	逐样本一致性、时延、通信量
协议层与控制面黑盒用例	17 类	异常输入、重放与并发探针验证	首个拦截层、兜底层级、资源状态

4.2 离线训练与部署准备实验

本节围绕离线部署准备链路组织四类受控实验，分别对应蒸馏补偿、部署对齐控制、`base_rate` 扫描以及阈值搜索与校准。本节结论仅代表离线部署准备阶段的受控证据，而不代表完整收敛训练的最终上界。

在这一口径下，本节重点回答三个问题：蒸馏补偿是否改善部署后可用边界，

secure_static_train_depth 这类部署对齐约束在当前控制实验中呈现何种收益形态，以及正式交付 bundle 所采用的 base_rate、阈值搜索与离线校准配置是否确实来自统一的部署准备流程。

4.2.1 蒸馏补偿配对实验

表 4-2 蒸馏补偿配对实验（统一 warm-start、统一 1 epoch）

方案	无蒸馏基线	蒸馏补偿	增量
cls_distill weight	0	1.0	-
token_distill weight	0	0.02	-
argmax_accuracy (%)	74.2366	77.4809	+3.2443
threshold_accuracy(%)	90.2672	90.8397	+0.5725
AUC	0.9549	0.9575	+0.0026
结论	作为离线部署准备的参考起点	argmax、阈值精度与 AUC 均优于无蒸馏基线	说明蒸馏补偿对部署口径有正向收益

表 4-2 对应的实验目的是检验蒸馏补偿是否属于部署准备的必要步骤，而非单纯的训练技巧。在统一 warm-start、统一 524 张验证集、统一 1 epoch 的条件下，蒸馏补偿方案相较无蒸馏基线在 argmax 精度、阈值精度和 AUC 上分别提升 3.2443、0.5725 和 0.0026。该结果支持“蒸馏补偿有助于稳定部署边界”这一结论，但并不等同于已经完成更长训练周期下的最终最优性证明。

4.2.2 部署对齐控制实验

表 4-3 对应的实验目的是检验 secure_static_train_depth 这类部署对齐约束在当前控制设置下的实际收益。结果显示，`secure\_static\_train\_depth=12` 虽使 argmax 精度提高 4.0076 个百分点，但阈值精度下降 1.5267 个百分点，AUC 下降 0.0117。因此，该实验支持“部署对齐控制已完成单因子验证”，但当前证据只呈现混合或中性收益，不能写成已显著优于对照。

表 4-3 secure_static_train_depth 部署对齐配对实验（统一 warm-start、统一 1 epoch）

方案	depth=0	depth=12	增量
secure_static_skip pruning	true	true	-
secure_static_train depth	0	12	-
argmax_accuracy (%)	77.4809	81.4885	+4.0076
threshold_accuracy (%)	90.8397	89.3130	-1.5267
AUC	0.9575	0.9458	-0.0117

结论	作为 secure-static 对齐的参考配置	argmax 提升, 但阈值精度与 AUC 未形成优势	当前 1 epoch 控制实验下收益呈混合状态
----	--------------------------	-----------------------------	-------------------------

4.2.3 base_rate 扫描与最终部署口径

如图 4-1 所示, base_rate 扫描用于确定三阶段 token 保留率的最终部署口径。服务器复核结果表明, 'base_rate=0.7' 在当前统一 warm-start、统一验证集和统一 1 epoch 扫描条件下取得最高阈值精度与 AUC, 因此正式 bundle 采用 '0.7 / 0.49 / 0.343' 作为三阶段保留率。这一结果说明, 离线阶段输出的是可部署参数与校准口径, 而不是单次检查点的经验选择。

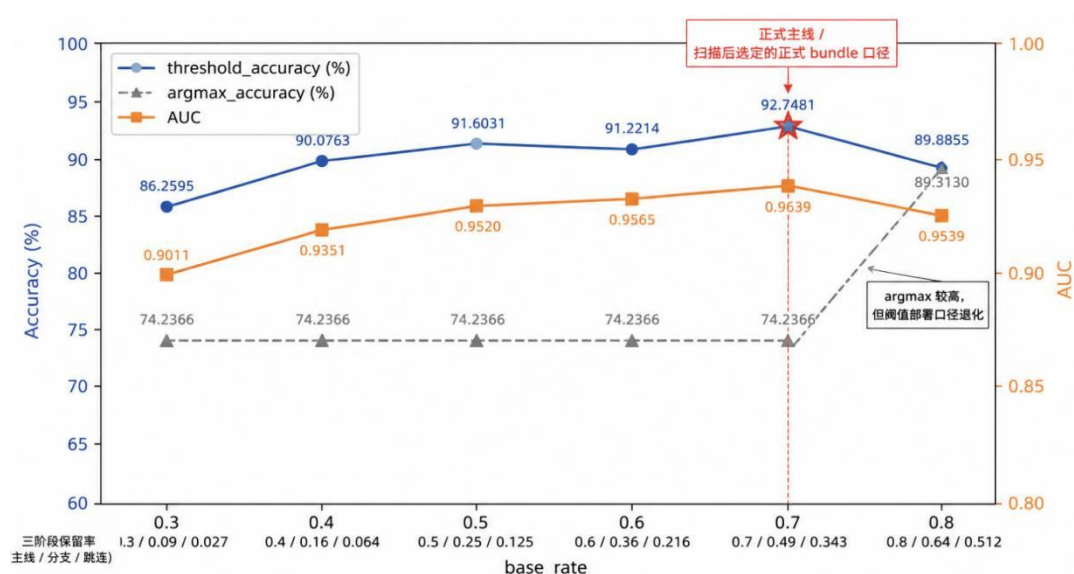


图 4-1 base_rate 扫描结果与最终部署口径

4.2.4 阈值搜索与离线校准链路

阈值搜索与离线校准的目的将部署边界在离线阶段固定下来。服务器结果文件表明，正式 bundle 中保留的默认阈值为 0.3577，但最终交付口径采用的是在 full-val 524 张验证样本上重新搜索得到的最佳阈值。

表 4-4 阈值搜索与离线校准链路结果

评估口径	dynamic fullval depth10	dynamic fullval raw
default_threshold_in_bundle	0.3577	0.3577
best_threshold	0.6620	0.6226
argmax_accuracy (%)	74.2366	74.6183
best_threshold_accuracy (%)	92.7481	92.7481
AUC	0.9639	—
说明	depth10 正式校准口径，服务于在线 2PC 部署	raw summary，用于对照 depth10 校准链路

表 4-4 中的`dynamic fullval depth10`与`dynamic fullval raw`两行分别对应 depth10 部署口径与未加 depth 约束的原始校准摘要。前者给出`best\_threshold=0.6620`、`threshold\_accuracy=92.7481%`和`AUC=0.9639`，说明冻结 bundle、阈值搜索与 AUC 参考链已经形成可回溯闭环；后者则作为对照，表明正式部署口径来自统一离线校准链路，而非任意选取。

4.3 基线对比

本作品与不同基线方案在医疗任务上的性能对比结果如图 4-2 所示。从同仓与外部对比结果看，本作品医疗动态安全剪枝链路相较固定结构静态对照线提升 3.0534 个百分点，相较当前仓内可复现的原始 DynamicViT 基线提升 17.3664 个百分点，说明协议友好重写后，动态路径仍保留了可用于部署评估的判别能力。另一方面，与同数据集强明文参考 MPCViT（面向多方计算的视觉 Transformer）相比，本作品当前仍存在 4.2366 个百分点的阈值精度差距；与同架构静态上界 DeiT-S（数据高效图像

Transformer 小型模型）相比，差距为 3.2419 个百分点。这些差距说明本作品的价值主要在于“双向隐私+动态剪枝安全执行”的系统能力，而不是在明文精度指标上全面超过所有强明文参考。

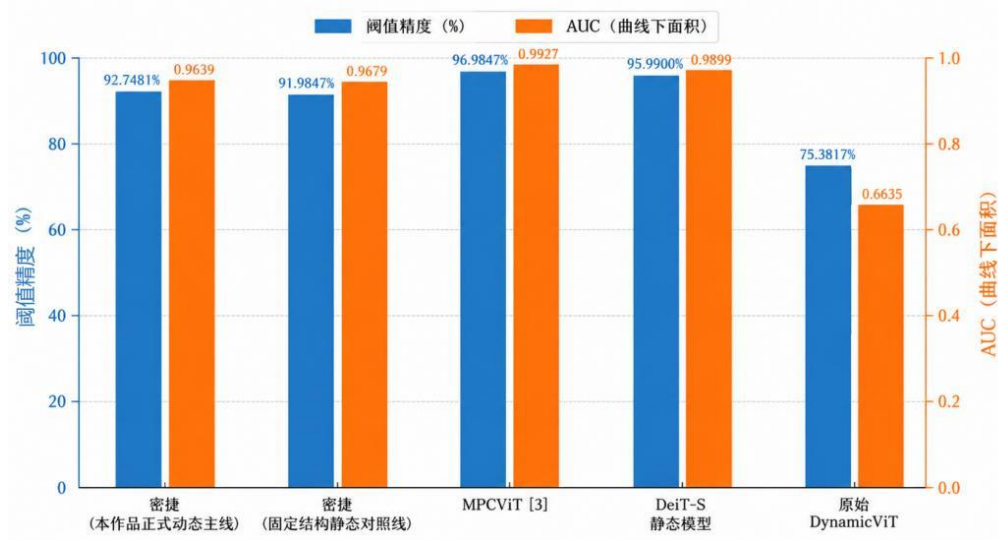


图 4-2 不同方案医疗任务性能基线对比

需要说明的是，外部对比表中“原始 DynamicViT”一行现统一采用当前仓内与服务器均可复现的原始明文基线口径。此前遗留的“20 epoch/80.73%”旧值未能在最终交付仓与服务器环境中追溯到配套权重文件，因此本次修订不再继续引用该旧数值。

4.4 医疗主要交付场景结果

医疗主要交付场景下的核心精度、性能与隐私边界指标如表 4-5 所示。图 4-3 对比了医疗动态安全剪枝路径与 DenseNet121[40]明文卷积基线的概率分布及阈值位置。结合 argmax 精度与阈值精度可以看出，当前动态路径的主要问题并非样本排序能力完全丧失，而是部署边界相对固定结构参考发生了系统性偏移。故而，针对动态路径单独校准部署阈值，是当前交付方案中的必要步骤。

从部署评估角度看，阈值校准用于将动态路径保留下来的排序能力转化为稳定的最终判决边界。当前结果表明，医疗动态安全剪枝链路在 AUC 维度仍保留较强样本区分能力，但未校准 argmax 输出与最终阈值判决并不完全等价。因此，报告将 AUC、argmax 精度和阈值精度分开呈现，以避免把未校准输出误读为最终部署效果。

表 4-5 医疗主要交付场景核心指标结果

指标	结果	说明
全量验证集 argmax 精度	74.2366%	反映动态路径在默认决策边界下的直接分类结果
全量验证集 阈值精度	92.7481%	基于 524 张验证样本完成统一阈值搜索后的正式指标
全量验证集 AUC	0.9639	服务器按同口径重跑补齐
部署阈值	0.6619606018	动态路径单独校准后的正式部署阈值
32 条样本验证 批次平均时延	48.69 秒/样本	当前VPS原始代码复跑 总时长1558.20秒
32 条样本验证 批次通信量（TX单计）	68.38 GiB	当前VPS环回接口TX 增量单计
32条复跑样本 阈值精度/AUC	93.75% 0.96484	当前VPS原始代码 固定32条医疗样本

因此，医疗主交付场景中的结果解释应分成“排序能力是否仍在”和“最终部署边界是否已经稳定”两个层次来读。AUC 较高说明动态路径在样本区分上仍保留了主要判别信息，阈值精度则进一步说明经过单独校准后，这种排序能力已经能够映射为可交付的最终决策效果。将两个层级的工作拆分说明，就能有效区分未经校准的原始输出，和最终落地的部署判决结果，避免二者混淆。

同口径服务器复核表明，医疗动态安全剪枝链路在 full-val CPU runtime-pruning reference 口径下的 AUC 为 0.9639。这说明当前动态路径并未失去样本排序能力，影响部署效果的关键问题仍然是决策边界相对固定结构参考发生系统性偏移，因此动态路径单独阈值校准仍是正式部署中的必要步骤。

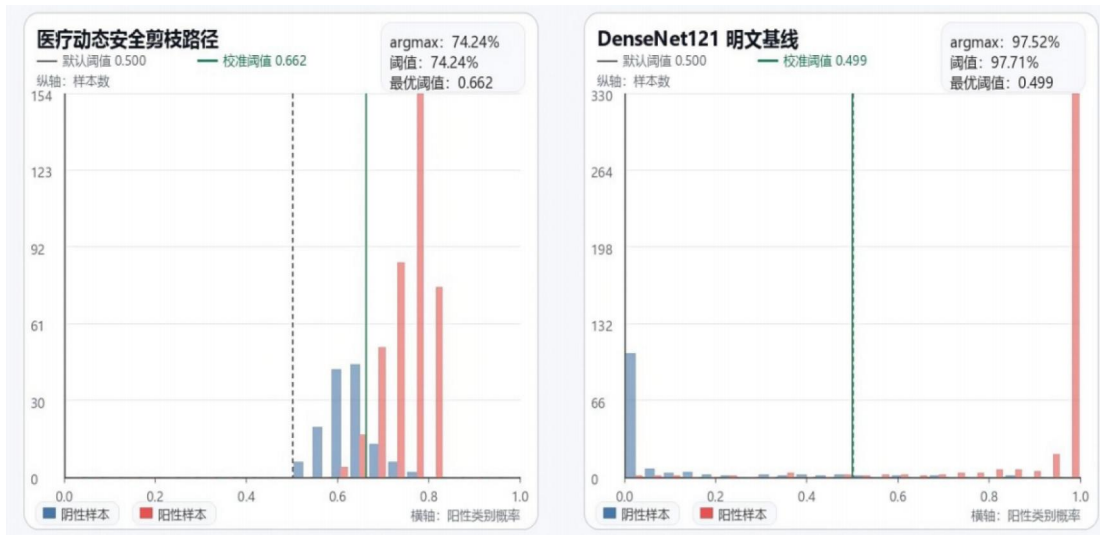


图 4-3 动态安全剪枝与 DenseNet121 概率分布对比

4.5 性能与通信量分析

医疗交付场景与金融边界压力验证场景的端到端性能与通信量统计结果如图4-4所示。医疗数据为当前VPS原始代码复跑结果，通信量按环回接口TX增量单计；金融数据为原报告历史记录。因环境与计数方法不同，两组数据不作直接比例比较。

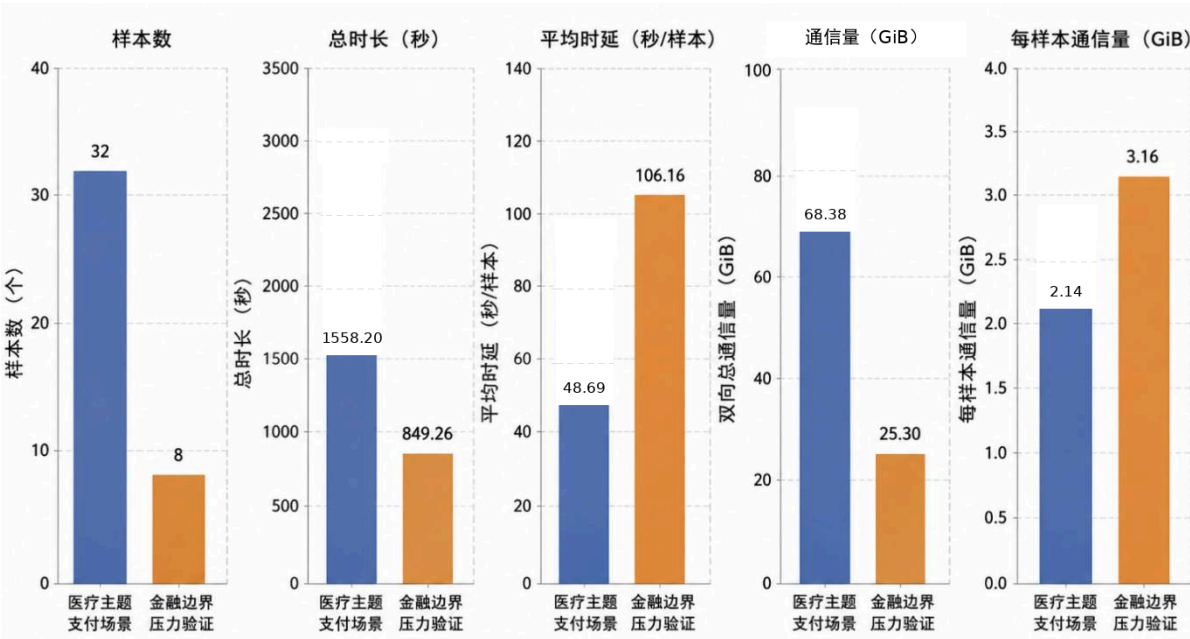


图 4-4 不同场景端到端性能与通信量统计

为进一步量化算子优化的收益，本作品补充了统一基准下的算子级代理对比，结果如表 4-6 所示。

表 4-6 算子级代理基准对比结果

代理基准 对比组	通信比值 (左/右)	时间比值 (左/右)	适用范围说明
同形状算子 代理基准	0.149x	0.528x	同一模型形状、同一 2PC benchmark harness； 主要用于观察算子配置导致的安全开销差异。
异构结构代 理基准	16.845x	2.955x	同一 MPCFormer local 2PC benchmark harness；各 profile 使用各自模型结构参数。 这不是 full-val 医学图像 pipeline 通信量。

注："同形状算子代理基准"基于统一 DeiT-S 模型结构测试；"异构结构代理基准"仅用于说明模型结构差异对性能的影响，不代表医疗端到端链路性能。

4.5.1 代价分解与代理基准

为便于统一表达时延与通信量的来源，本节先用式（4-1）至式（4-2）给出端到端代价的分解形式，再结合当前原型链路的实测结果解释主要瓶颈。

$$T_{\text{total}} = T_{\text{pre}} + T_{\text{guard}} + T_{\text{SPU}} + T_{\text{post}} \quad (4-1)$$

$$(V_{\text{total}} = \sum_{r=1}^R (V_{\text{send}}^{(r)} + V_{\text{recv}}^{(r)})), \rho_V = \frac{V_{\text{密捷}}}{V_{\text{ref}}}, \rho_T = \frac{T_{\text{密捷}}}{T_{\text{ref}}} \quad (4-2)$$

其中， T_{total} 由前端预处理、服务端控制面快检、SPU 密态执行与结果回传/审计收尾四部分组成； V_{total} 表示一次完整推理的双向总通信量， ρ_V 与 ρ_T 分别表示相对通信量与相对时延比。

性能证据分为两层：其一是上述端到端双向隐私运行结果，用于说明医疗动态安全剪枝链路和金融压力样本在当前原型中的总时延和总通信量；其二是统一安全基准下的算子级代理对比，用于说明算子替换本身的开销趋势。在固定同一 DeiT-S 形状的代理基准对比中，密捷的安全友好算子将通信压缩到外部基准算子的 0.149 倍，时间压缩到 0.528 倍，说明算子替换本身具有正向收益。另一方面，跨模型结构的异构结构代理基准只能用于说明结构尺度差异会显著放大绝对代价，不能与医疗全量验证链路混写。

为进一步说明本作品所采用算子改写在安全推理中的实际收益，本作品从同形代理基准、单函数网络敏感性以及低层预处理/共享转换三个层面，对系统开销进行了

分解测量。与仅给出端到端总时延不同，这种分解视角能够更清晰地解释通信瓶颈究竟来自高阶非线性、注意力归一化，还是来自共享表示转换本身。

在保持输入张量形状一致的前提下，本作品构造了两组代理路径进行同形比较：一组为传统的 ReLU（Rectified Linear Unit）[41]+Softmax，另一组为当前作品中更接近部署思路的 Quad+Softmax_2QUAD。测试结果如表 4-7。

表 4-7 代理路径比较结果

代理路径	平均耗时/s	平均通信量/MiB
ReLU+Softmax	15.1696	5918.69
Quad+Softmax_2QUAD	8.0973	881.05

相较之下，当前改写路径在同形条件下实现了约 46.62%的时延下降与约 85.11%的通信量下降。这说明，在匹配输入张量形状的算子级代理基准下，安全友好化改写能够降低非线性与归一化路径的通信和时延开销。需要强调的是，该结果用于解释算子替换带来的局部收益，不等同于完整医疗端到端链路的总性能提升。

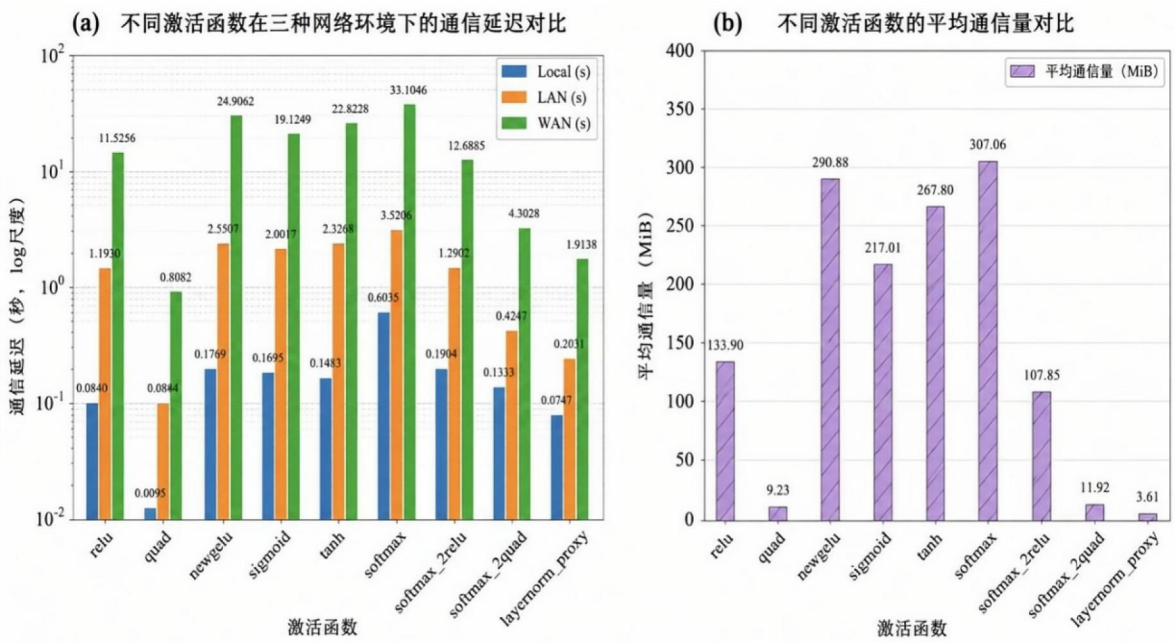


图 4-5 各安全函数不同网络敏感性与单次安全执行的平均通信量

为进一步考察不同网络环境下各安全函数的敏感性，本作品在相同输入规模下，

分别于本地回环（Local）、局域网仿真（LAN）与广域网仿真（WAN）条件下，对代表性函数进行了单独测量。测试中，激活类算子输入张量规模统一为[1,197,1536]，注意力归一化类算子输入张量规模统一为[1,6,197,197]。从通信量角度看，不同函数之间的差异比纯本地算力差异更为显著。特别是在 WAN 条件下，低通信量函数的收益会被网络时延进一步放大。结果如图 4-5 所示。

仅从高层算子角度分析仍不足以完全解释系统的通信来源。为此，本作品进一步在 CrypTen 的 provider 与 converter 层面对低层原语进行了分解测量，分别统计 Beaver triple、binary triple、B2A 随机材料以及 A2B/B2A 共享转换的代价。需要指出的是，当前实现采用 TFP provider，在该机制下 additive/binary/B2A_rng 材料主要通过本地相关性生成完成，因此在线测量中通信量为 0；真正显著消耗网络带宽的是共享表示转换过程本身。

由图 4-6 结果说明，在当前两方安全推理链路中，预处理材料的生成与分发并不是主导瓶颈，真正对跨网络时延产生决定性影响的是 A2B/B2A 等共享表示转换操作。换言之，系统优化不应仅停留在“减少乘法”这一层面，还应尽量减少需要跨算术域与布尔域反复切换的比较与非线性子过程。这也解释了为什么本作品采用 fixed_square、uniform attention 与 softmax_2quad 等改写后，能够在 WAN 条件下取得比 Local 条件下更显著的收益。

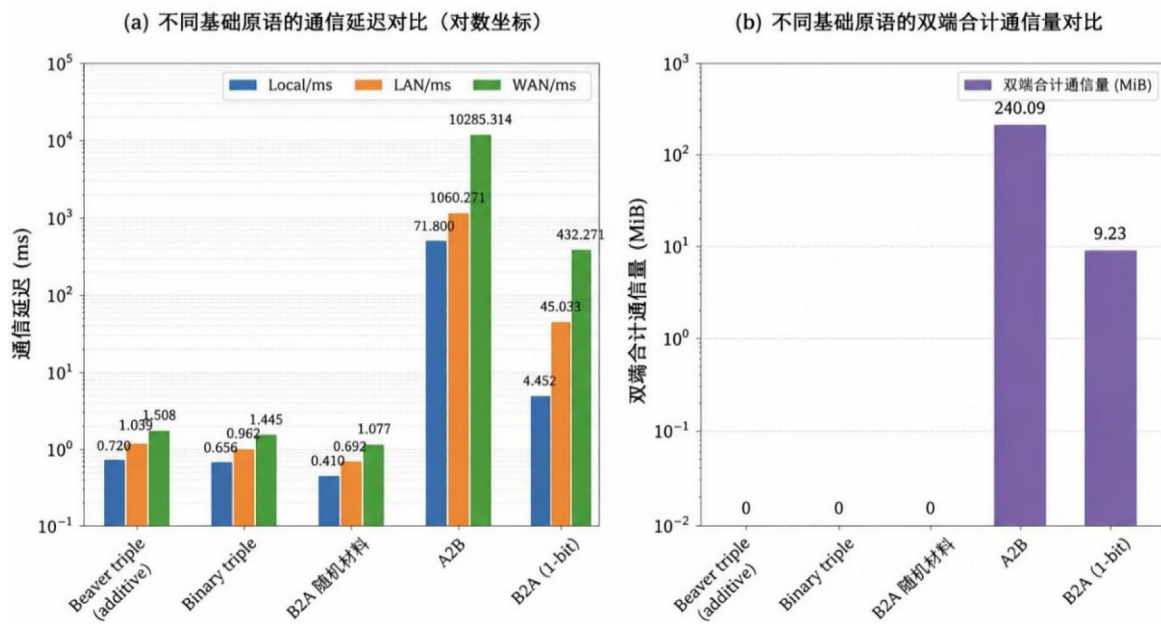


图 4-6 不同基础原语测试结果对比

4.5.2 安全比较网络的复杂度与通信量推导

本作品在固定图安全化剪枝中采用带索引跟踪的编码键双调排序网络（bitonic sorting network）[35]完成 token 保留边界求解。该设计不同于“先恢复阈值、再对所有 token 逐项比较”的两阶段写法，而是直接对屏蔽后的 token 得分进行稳定排序，并由排序后的前 K 个索引构造 keep-mask。由此，本节统一以 bitonic sorting network 为对象分析其复杂度与通信量。

设某一剪枝阶段的输入 token 数为 N ，对应的保留预算为 K 。由于标准 bitonic sorting network 通常要求输入长度为 2 的幂，本作品首先将输入补齐到

$$N_{\text{pad}} = 2^{\lceil \log_2 N \rceil}.$$

记 $m = \log_2 N_{\text{pad}}$ ，则 m 表示排序网络的层级规模。对于长度为 N_{pad} 的 bitonic sorting network，其同步深度为

$$D(N_{\text{pad}}) = \sum_{t=1}^m t \frac{m(m+1)}{2} = O(\log^2 N_{\text{pad}}).$$

其中， $D(N_{\text{pad}})$ 表示安全比较交换需要经历的最大同步轮次数。由于安全比较通常涉及布尔共享、比较门或共享转换，网络深度直接影响在线阶段的交互轮次和等待时间。

相应地，比较器总数为

$$C(N_{\text{pad}}) = \frac{N_{\text{pad}}}{2} \sum_{t=1}^m t = \frac{N_{\text{pad}} m(m+1)}{4} = O(N_{\text{pad}} \log^2 N_{\text{pad}}).$$

其中， $C(N_{\text{pad}})$ 表示整个排序过程中需要执行的 compare-and-swap 单元数量。若单个安全比较交换的平均通信代价记为 c_{cs} ，则单阶段排序通信量可近似表示为

$$T_{\text{sort}}(N_{\text{pad}}) = C(N_{\text{pad}}) \cdot c_{\text{cs}}.$$

进一步代入比较器数量，可得

$$T_{\text{sort}}(N_{\text{pad}}) = \frac{N_{\text{pad}} m(m+1)}{4} \cdot c_{\text{cs}} = O(N_{\text{pad}} \log^2 N_{\text{pad}}) \cdot c_{\text{cs}}.$$

当前实现的关键优化在于：排序完成后直接读取前 K 个索引构造 keep-mask，而不再额外执行一次全量阈值比较扫描。若采用“排序求阈值—全量比较生成掩码”的两阶段写法，还需要对所有 N_{pad} 个 token 执行一次额外比较，其通信量可写为

$$T_{\text{scan}}(N_{\text{pad}}) = N_{\text{pad}} \cdot c_{\text{cmp}},$$

其中 c_{cmp} 表示单次安全阈值比较的平均通信代价。则两阶段写法的总通信量近似为

$$T_{\text{two-stage}} = T_{\text{sort}}(N_{\text{pad}}) + T_{\text{scan}}(N_{\text{pad}}).$$

而本作品当前实现的通信量为

$$T_{\text{ours}} = T_{\text{sort}}(N_{\text{pad}}) + T_{\text{mask}},$$

其中 T_{mask} 表示由前 K 个索引构造 keep-mask 的代价。由于该步骤主要依赖索引选择和掩码写入，其通信开销通常小于额外全量阈值比较扫描。因此，相较两阶段写法，本作品至少避免了

$$\Delta T \approx N_{\text{pad}} \cdot c_{\text{cmp}} - T_{\text{mask}}$$

这一部分额外通信开销。

对本作品当前医疗主线而言，输入分辨率为 224×224 ，patch size 为 16，因此初始空间 patch token 数为

$$N_0 = \left(\frac{224}{16}\right)^2 = 196.$$

若只考虑空间 token，并忽略 class token 对剪枝预算的影响，则在正式部署口径 base_rate=0.7 下，三个剪枝阶段的目标保留率为 (0.7, 0.49, 0.343)。

对应保留预算约为

$$K_3 = \lceil 196 \times 0.7 \rceil = 138,$$

$$K_6 = \lceil 196 \times 0.49 \rceil = 97,$$

$$K_9 = \lceil 196 \times 0.343 \rceil = 68.$$

这说明，动态剪枝并不是只在模型语义上减少冗余 token，也会使后续阶段的有效安全比较规模逐步收缩。

进一步地，若三个剪枝阶段的有效输入规模分别记为 N_3, N_6, N_9 ，则三阶段排序总通信量可写为

$$T_{\text{total}} = \sum_{l \in \{3, 6, 9\}} O(N_l^{\text{pad}} \log^2 N_l^{\text{pad}}) \cdot c_{\text{cs}}.$$

其中，

$$N_l^{\text{pad}} = 2^{\lceil \log_2 N_l \rceil}.$$

若后续阶段继承前序 keep-mask 并只对活跃 token 进行有效排序，则 N_6 与 N_9 会随前序剪枝逐步下降；若安全后端出于固定图编译要求仍保留完整张量形状，则有效比较可通过掩码屏蔽失活 token，但外部可见执行图仍保持不变。这也是本作品在复杂度分析中区分“固定图形状”和“有效活跃 token 规模”的原因。

综上，本作品的安全剪枝边界求解具有三个特点。第一，bitonic sorting network 的比较器数量为 $O(N \log^2 N)$ ，同步深度为 $O(\log^2 N)$ ，因此其主要通信和交互成本来自安全 compare-and-swap。第二，当前实现通过 encoded-key 排序和 top-K 索引回写直接生成 keep-mask，避免了额外一次全量阈值比较扫描。第三，随着动态剪枝逐阶段减少活跃 token，后续阶段的有效安全比较和掩码传播成本也会同步下降。因此，动态剪枝在密态推理中的作用不仅是减少视觉 Transformer 的语义冗余，也是在协议层面压缩后续安全比较网络的有效执行规模。

4.5.3 端到端瓶颈来源

当前时延与通信量偏高，主要由三类因素共同造成。第一，动态安全剪枝链路保留了词元剪枝的密态比较、排序与掩码化决策链，相比固定结构安全推理会引入额外的协议轮次。第二，当前运行口径是同机本地双节点（colocated localhost）2PC 原型链路，虽然两方节点位于同一服务器内，但张量交换、线性层通信与 Softmax 归一化指数函数相关算子仍构成主要代价来源，不能将 localhost 原型误解为“几乎无通信成本”。第三，当前 Web 演示后端优先保证正确性、边界防护和证据留存，尚未针对生产级吞吐进行异步化、流水化或任务队列优化。因此，本作品只保留已直接落盘的端到端结果与算子级代理证据，不额外给出未经观测的阶段占比、P50/P95 延迟或生产级并发吞吐承诺。

4.6 金融边界压力验证结果

扩展边界压力验证结果如表 4-8 所示。该部分不作为与医疗影像并列的主交付场景，主要用于观察同一安全执行链路在非自然图像输入映射下的稳定性边界。金融部分仅承担扩展压力测试职责，不用于证明本作品已经形成面向金融业务的完整交付系统。其价值在于验证控制面、动态路由和安全执行语义在另一类高敏感输入形式下是否仍能保持一致。

表 4-8 金融边界压力验证核心指标

指标	结果	说明
8 条压力样本逐样本一致性	8/8 样本差异为 0	与明文参考逐样本对齐
部署效率	106.16 秒/样本	原报告历史边界口径
双向总通信量	25.30 GiB	原报告历史计数口径
参数规模保留比例	68.39%	体现低秩压缩收益

4.7 协议层异常输入与鲁棒性验证

鲁棒性验证围绕协议入口、结构化元数据、密态分片和控制面状态四类边界展开，而不是仅以单一拦截率概括系统表现。协议层测试覆盖分块传输编码（Transfer-Encoding: chunked）、超大报文长度声明（Content-Length）、boundary 参数混淆、重复字段、畸形分片报文头、空字节注入、非空尾部附加数据、UTF-16 编码结构化元数据、超长结构化元数据以及截断请求体；控制面测试覆盖随机一次性标识（Nonce）并发重放、相同载荷更换 Nonce、同一来源地址并发占满与短窗限频。每个用例均记录首个拦截层、兜底层级、返回处置与资源状态回落摘要，以同时验证拦截有效性、资源状态回落和原型级可服务性。协议层与控制面鲁棒性验证的整体统计结果如表 4-9 所示。

进一步看，这 17 类用例覆盖了三类不同风险来源。第一类是协议语法层异常，例如 boundary 混淆、重复字段和截断请求体，用于验证原始报文解析与 MIME 结构树是否稳健；第二类是语义与张量层异常，例如 UTF-16 元数据、超长 JSON 和异常 share 载荷，用于验证结构化解析、哈希校验与张量快检是否联动；第三类是运行状态层异常，例如 nonce 重放、相同载荷更换 nonce 与来源地址并发占满，用于验证重放、限频和 inflight 守卫是否真正起效。因而，本节的价值不只在于“17/17 命中”，更在于各层拦截责任被清楚地区分出来。

之所以同时观察文件描述符、套接字文件描述符、常驻内存和线程数四类资源指标，是因为它们分别对应文件句柄、网络句柄、常驻内存与线程占用四个最容易在异常输入压力下失衡的资源面向。对当前原型而言，这组指标不能证明“绝对无资源成

本”，但足以证明异常处理完成后是否出现持续泄漏、是否存在无法回落的占用，以及是否需要把某类请求进一步前移阻断。也正因此，本作品在这一节中坚持使用“资源状态回落”而不是“零资源影响”作为结论表述。

表 4-9 协议层与控制面鲁棒性验证统计

测试组	用例数	按预期通过	FD/Socket 无泄漏	稳态回落用例
协议层异常输入	13	13/13	13/13	11/13
控制面守卫	4	4/4	4/4	1/4
合计	17	17/17	17/17	12/17

从当前留存结果看，协议层异常输入与控制面守卫共覆盖 17 类黑盒用例，17/17 均返回预期拦截或限制结果；全部用例在采样窗口内均未观察到文件描述符或套接字文件描述符的净增长。与此同时，部分并发与大负载探针会带来瞬时常驻内存抬升，因此本作品将其如实记录为“资源成本存在但句柄未泄漏”，而不是将其包装为零成本防护。

为进一步验证系统在面对异常输入与控制面扰动时的稳健性，本作品围绕协议层关键边界设计了异常注入与兜底验证实验，并将主要异常场景、首个拦截层级及系统响应结果整理如图 4-7 所示。各类异常场景的攻击特征与系统拦截层级明细以及对应异常场景的系统返回处置、资源状态与证据来源明细见附录 B。

异常场景	首个拦截层	结果状态	系统状态摘要
协议字段超长	协议解析层	通过	报文在解析阶段被拒绝，未进入后续处理流程，无状态变更。
非法字段值 / 枚举越界	协议解析层	通过	非法值被识别并丢弃，未影响会话上下文与业务处理。
分片畸形 / 重组异常	报文处理层	通过 (有瞬时资源代价)	分片被拒绝重组，释放相关资源，未触发异常告警风暴。
重放报文	会话层	通过	重放报文被识别并丢弃，会话保持稳定，无重复执行。
时序窗口异常	时序校验层	通过 (有瞬时资源代价)	超出窗口的报文被限流处理，轻微资源波动后迅速恢复。
协议状态机非法跳转	状态机引擎层	通过	非法状态跳转被阻断，状态机保持一致性，未进入异常态。
认证流程异常	认证鉴权层	通过	认证失败路径被拦截，未授予权限，未建立受信会话。
控制帧洪泛	流量控制层	通过 (有瞬时资源代价)	触发限流与速率控制，短时资源占用上升后回落至基线。
扩展选项异常	协议合规层	通过	异常选项被忽略或拒绝，维持协议合规性与兼容性策略。
未知协议 / 非预期报文	协议识别层	通过	未知协议被隔离处理，未影响正常协议识别与转发。

说明：结果状态为“通过”表示在该异常场景下系统被有效拦截并保持稳定；“通过 (有瞬时资源代价)”表示存在可控的瞬时资源消耗或短时抖动，但系统未失稳且可自恢复。
 验证方法：黑盒测试为主，结合白盒辅助分析与日志审计，覆盖协议解析、会话管理、状态机、认证鉴权、流量控制等关键控制制面能力。

图 4-7 协议层异常输入与控制面兜底验证矩阵

资源状态采用可观测代理指标描述，而不作无法直接证明的绝对化承诺。本作品将“系统仍可服务”限定为：请求结束后 `inflight` 占用计数回落，文件描述符数量与套接字文件描述符数量回到基线附近，未观察到连接句柄持续泄漏；常驻内存在部分已通过前置校验的压力用例中出现瞬时抬升，但重复异常注入测试后未观察到单调持续增长，因此仅将其表述为原型阶段的瞬时资源成本，而不宣称所有指标完全归零。

4.8 结果分析

4.8.1 结果结论与工程意义

综合上述结果，可以得到三点结论。第一，动态安全剪枝链路在当前两方安全推理原型中保留了可校准的医学图像判别能力，并未退化为单纯的固定结构安全推理。第二，算子级代理基准表明 `fixed_square`、`uniform attention` 等安全友好函数能够降低局部通信与时延开销，但完整端到端性能仍受模型规模、协议轮次与 2PC 执行方式共同制约。第三，协议层异常输入与控制面探针能够在请求进入 SPU 之前完成前置阻断，并未观察到文件描述符或套接字句柄的持续泄漏，但这一结果只支持原型环境下的可服务性，不等同于生产级抗压能力证明。

将本作品置于现有私有 Transformer 推理研究中考察，其当前结果更接近于“在医疗视觉任务中建立动态剪枝双向隐私闭环”的工程验证，而非单纯追求固定结构模型的最低时延。已有公开结果表明，协议级优化、非交互执行或密态渐进剪枝都可能显著改善绝对效率；相比之下，本作品更强调在浏览器本地分片、服务端权威快检和两方安全执行同时成立时，动态剪枝边界仍可被保留并校准。

4.8.2 对比口径与评价边界

因此，结果解释需要区分三类比较口径：同仓动态链路与静态安全基线的比较，用于判断动态剪枝安全化改写是否带来部署收益；同数据集强明文参考的比较，用于说明当前安全化改写距离普通明文模型仍有多大差距；外部私有推理系统或论文结果的比较，用于定位技术路线，而不应与本作品端到端 Web 控制面和本地双节点 SPU 原型直接混作数值结论。

4.9 消融实验与必要性验证

4.9.1 动态剪枝必要性

为验证各关键设计的必要性，本节分别从动态剪枝路径、安全友好函数组合与控制面边界三条主线开展消融分析。图 4-8 给出了医疗动态安全剪枝链路、去掉动态剪

枝后的静态安全基线以及原始明文参考基线的同口径对照结果。在统一 524 张验证集上，动态安全剪枝链路的阈值精度达到 92.7481%，较静态安全基线提升 3.0534 个百分点，AUC 由 0.9539 提升至 0.9639；与此同时，其 `argmax` 精度并未高于静态安全基线。这说明动态剪枝安全化改写的主要价值不在于未校准条件下的直接分类，而在于部署口径下更稳定、更可校准的判别边界。

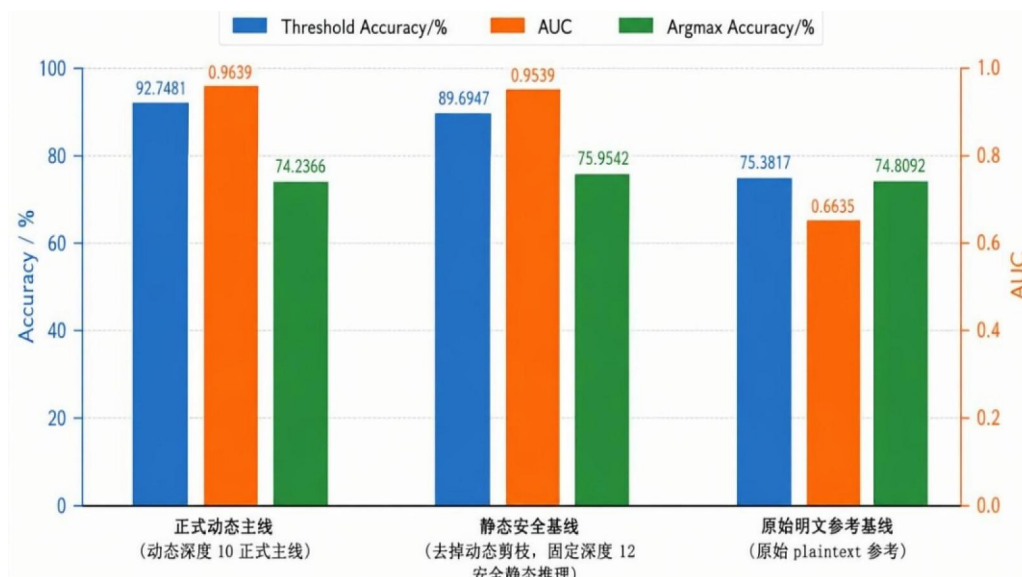


图 4-8 动态剪枝消融结果（524 张验证集，同口径）

进一步地，与原始明文参考基线相比，医疗动态安全剪枝链路的阈值精度提升 17.3664 个百分点，表明本作品在动态剪枝边界、固定图安全语义和部署后阈值校准之间形成了协同增益。

4.9.2 安全友好函数联合消融

为避免仅凭算子级代理 `benchmark` 讨论函数改造收益，本节进一步引入服务器上的联合配置消融结果。该实验直接对比正式安全友好主线与“撤除正式函数改写组合”后的候选 `run`，因此回答的是 `fixed_square` 与 `uniform attention` 这组正式部署函数族的联合贡献，而不是某个单独函数的独立贡献。

表 4-10 所对应的联合消融来自服务器。与官方配对 `run` 相比，该候选在命令末尾额外覆盖了 `--use_square_gelu false` 与 `--use_approx_attn false`，即撤除了正式主线中的 `fixed_square+uniform attention` 组合。因此，正文应将其解释为“去掉正式安全友好函数组合后的联合影响”，而不能将下降幅度直接归因于单一函数。

表 4-10 安全友好函数联合消融

方案	正式安全友好主线	联合消融	增量
函数组合	fixed_square+uniform attention	关闭`use_square_gelu`与`use_approx_attn`	-
argmax_accuracy(%)	74.2366	69.8473	-4.3893
threshold_accuracy(%)	92.7481	88.1679	-4.5802
AUC	0.9639	0.9111	-0.0528
结论	作为当前正式交付口径	撤除正式函数组合后判别边界明显退化	说明安全友好函数组合对当前部署主线不可缺

第五章创新性与局限性

5.1 算法层创新

现有问题在于：原始 DynamicViT 的词元删除、阈值比较与数据相关 Top-K 选择依赖明文控制流，直接迁入安全计算图会同时遭遇变长结构、路径泄露和并列分数不稳定三类困难，因此许多安全推理方案会优先采用固定结构执行图，以降低协议实现复杂度，但这也会削弱动态 token 管理带来的效率优化空间。

该设计的做法是把 pruning boundary 从“删除式表达”重写为“掩码化表达”，将词元保留决策改写为基于安全选择原语的掩码更新，将阈值判断改写为基于安全比较原语的边界判定，并结合编码键双调排序（Encoded-Key Bitonic Sort）处理安全 Top-K 选择与并列分数决断问题。这样做的意义在于，不仅让动态剪枝能够进入固定图安全执行环境，而且保留了按样本变化的剪枝决策能力，而不是用一个固定结构模型替代原始方法。

与“先求阈值再逐项比较”的常见表达不同，本作品选择通过编码键双调排序直接恢复前 K 个索引，再据此构造 keep-mask。这样做的工程优势在于：一方面可减少二次全量密态比较带来的额外通信，另一方面能够在并列分数情形下保留稳定索引顺序，使动态路径结果在重复运行、审计复核与部署复现实验中保持一致。

从算法创新边界看，这里的关键并没有把排序网络本身当作新协议，而是将排序网络放在正确的位置上：它既承担并列分数稳定化，又承担保留边界恢复，从而把原本分散在“阈值求解—逐项比较—外部回排”三个阶段的逻辑压缩为一条更适合固定图执行的密态路径。这种写法的直接收益是通信链路更短、重复运行更稳定，也更容易与后续审计和复现实验对齐。

如果进一步与常见动态剪枝或固定结构私有推理路线比较，可以看出本作品算法层创新有两个同时成立的判别标准。其一，它没有把“动态”简化成外部预先决定的明文路径，而是在 keep-mask、比较和排序三处共同保留样本相关边界；其二，它也没有为了适配安全执行而完全丢弃原始 DynamicViT 的逐 stage 语义，而是尽量保持训练侧 token_ratio、pruning_loc 和 PredictorLG 判别逻辑在部署侧仍然可追踪。正因如此，这一层创新既不是单纯的模型压缩，也不是单纯的协议替换，而是两者之间的结构化改写。

从可验证性角度看，算法层创新之所以能够成立，还因为它不是停留在“理论上可改写”的状态，而已经在仓内形成了从训练、参数抽取、静态前向、SPU secure pruning 到消融结果的连续证据链。训练主线保留 DynamicViT 语义，`static\_vit\_params.py` 固化 PredictorLG 参数和 token_keep_counts，`spu\_static\_vit.py` 执行安全前向与 keep-mask 构造，而第 4 章消融又进一步证明，去掉动态剪枝后阈值精度和 AUC 都会下降。因此，这一算法层贡献不仅有机理解释，还有运行实现和结果证据同时支撑。

这一层的技术难点在于：既要避免数据相关分支泄露，又要在固定图语义下近似保留原始动态模型的判别边界；同时，排序与并列分数处理必须在安全执行环境内稳定完成，否则最终决策链会因 tie 或索引不稳定而失去可验证性。

实验证据表明，该改写并未使动态路径退化为不可部署状态：动态安全剪枝链路相较固定结构静态对照线的阈值精度提升 3.0534 个百分点，相较未经安全化边界重写的原始 DynamicViT 提升 17.3664 个百分点。该内容由第 2.6 节密态改写、第 4.3 节基线对比、第 4.4 节医疗结果与第 4.8 节结果分析共同支撑。第 4.9 节的消融结果进一步说明，该创新实质性保留了动态剪枝在部署口径下的有效判别边界：去掉动态剪枝后，阈值精度由 92.7481% 下降至 89.6947%，AUC 由 0.9639 下降至 0.9539，表明动态剪枝安全化改写对最终可部署性能具有直接贡献。

5.2 系统层创新

现有问题在于，不少安全推理系统虽然使用了密态执行框架，但会在外部明文环境中预先决定剪枝路径，再把结果送入密态推理后端。这类做法虽然降低了工程难度，却使动态状态、路径信息甚至部分模型行为暴露在安全执行域之外，本质上仍属于半隐私运行。

本作品的做法是将动态剪枝预测器 PredictorLG、阈值判断与并列分数决断保留在安全执行域内，并以 SPU 与 OpenBumbleBee 组成两方安全计算链路，对输入数据、模型参数和中间动态决策状态同时进行保护。这样，系统不再依赖“外部先算路径、内部只回放”的简化策略，是在双向隐私前提下完成完整的动态决策链执行。

进一步地，这一系统层创新并不只体现在单机 colocated 原型里，还体现在角色分离意识已经前移到接口和配置层。当前仓库保留了医院侧、AI 侧和协调侧三类最小网关角色，以及 split manifest、单方 share 接收、远程 2PC 配置生成和单方节点启动入口，使“谁生成 share、谁持有模型、谁负责任务编排”在工程实现中不再是抽

象设想，而已经具有明确的参数、脚本和 API 边界。也正因为如此，本作品的系统层价值不只在完成一次安全推理运行，还在于把后续跨机构迁移时最容易混淆的角色责任提前结构化。

当前仓库把医院方与模型提供方的职责写在文档说明与 manifest、节点启动器和网关接口的组织方式中。医院侧可接收原始 PNG/JPEG 并生成 P1/P2 share，AI 侧只接收 P2 share 与模型 manifest，协调层则只暴露公开任务状态和 runner 编排信息。这种“先在工程接口上拆分责任，再讨论真实生产迁移”的做法，使系统层创新不再停留在架构图上，而在仓库操作入口中已经可以被实际检查到。

这一系统层设计的实质意义在于，动态路径被视为与输入、参数同等需要保护的执行状态。只有把预测器、比较器与保留决策统一保留在安全执行域内，系统才能真正声称自己同时保护了输入、模型和动态决策边界；否则，即便最终分类头在密态中运行，剪枝路径本身仍可能泄露样本难度、局部显著区域或模型偏好。

这一层的技术难点在于：既要在两方半诚实模型下维持输入与参数的双向隐私边界，又要让动态路径的关键控制流保留在密态环境内执行；如果处理不当，系统很容易退化为只保护输入或只保护模型的单向隐私方案。

实验证据方面，当前VPS原始代码复跑32条样本，平均时延为48.69秒/样本，环回接口TX增量单计通信量为68.38 GiB；该32条医疗样本阈值精度为93.75%，AUC为0.96484。服务端不接收明文像素值、模型参数不以明文暴露，对外仅返回最终分类结果；金融边界压力样本8/8与明文参考逐样本一致，性能数据保留原报告历史口径。该层内容由第2.1、2.5、3.2、4.4与4.6节共同支撑。

5.3 工程层特色与可验证交付能力

现有问题在于，很多 Web 默认将输入视为天然可信，缺少协议层异常输入阻断、审计链、资源状态回落记录和可复现实验入口。

本作品的做法是把浏览器工作线程、本地数据质量评估、审计哈希链、服务端权威快检、协议层异常输入模糊测试、重放与并发守卫一起纳入交付范围。这里的重点是使系统不仅能给出分类结果，还能证明输入经过了哪些检查、异常请求在何处被阻断，以及资源状态是否回到稳态。

从工具链完备性看，当前仓库不仅保留了展示站与控制面实现，还保留了与正式口径对应的最小启动、结果核对与边界验收工具：`start\_showcase\_spu\_demo.py`负责

重建当前展示链路，`showcase\_protocol\_fuzz.py`与`showcase\_guard\_stress.py`负责新展示接口的运行时验收，`split\_gateway\_smoke.py`负责本地演练医院/AI/协调三角色闭环，而`docs/evidence/README.md`与`results/README.md`则把正式数字和证据来源对齐到仓内固定文件。这样形成是为了有一套可重复启动、可重复验收、可重复核对结果的工程交付面。

由此，本作品把前端控制面和服务端快检纳入工程层特色讨论，是为了强调安全推理作品的交付边界必须同时包含输入可信性、异常请求阻断与证据留存机制。

从可核验性角度看，这一工程层特色还有一个直接优势：展示站、证据链和正式结果文件之间不存在互相脱节的“演示专用数字”。根 README、`results/README.md`、`docs/evidence/README.md`和报告正文共同引用的是同一批冻结 JSON；展示图件则由正式报告主文档抽取生成，而不是另外手工维护一套说明稿。因此，评审如果顺着仓库入口核查，能够把页面、图件、证据索引和正式数字连成一条连续链路，这正是工程交付可验证性的关键。

具体而言，根 README、`results/README.md`、`docs/evidence/README.md`与报告正文共同把医疗阈值精度、AUC、通信量和鲁棒性结果固定到同一批最终 JSON；`showcase\_extract\_report\_assets.py`又把报告图件与章节摘要抽取到展示站所需目录中，避免页面再维护另一套口径。因此，本作品的工程层可验证性并不依赖人工解释，而是依赖“同一数字在报告、仓库索引、展示入口三处可以互相回指”这一组织方式本身。

这一层的技术难点在于：前端要在不阻塞主线程的前提下完成高开销预处理与审计摘要生成，服务端要在进入 SPU 前完成原始报文、JSON 元数据、share 字节流和张量层的多级快检，同时还要把异常输入、重放与并发探针的结果转化为可复核的证据文件。

实验证据方面，本作品已覆盖 17 类协议层与控制面黑盒用例，17/17 按预期返回，且采样窗口内未观察到文件描述符或套接字文件描述符净增长；同时仓内提供 10 - 20 分钟最低可复现路径，用于复核完整计算链路。该内容层由第 3.3 节前端控制面、第 3.4 节服务端权威快检与审计、第 4.7 节鲁棒性矩阵、第 6.1 节复现路径和`README\_REPRODUCE.md`共同支撑。

5.4 与常见方案对比

表 5-1 本作品与常见隐私推理方案能力对比

对比对象	常见做法	本作品差异
固定结构安全推理	放弃动态剪枝，直接 改为固定结构模型	保留按样本变化的剪枝决策，并 把边界重写为可密态执行语义
明文剪枝+密态推理	外部明文环境先决定路径， 密态后端仅回放结果	剪枝决策链保留在安全执行域 内完成，避免半隐私运行
单向隐私推理	只保护输入或只 保护模型参数	同时保护输入数据、模型参数和 动态中间状态
普通 Web 演示	只展示结果页面，不提 供输入守卫与异常验证	加入服务端快检、审计链、模糊 测试与重放/并发守卫证据

表 5-1 比较了本作品与常见隐私推理方案在核心能力上的差异。该比较表明，本作品的重点不在于再复现一条固定结构安全推理链，而在于同时处理动态剪枝保留、双向隐私成立、动态决策链不外泄以及工程交付可复核四个问题。以 SViT[19] 为代表的方案更偏向固定结构 ViT 的协议优化，而本作品更偏向 DynamicViT 动态结构的安全化改写与端到端闭环构建。

图 5-1 进一步将固定结构安全推理、明文剪枝后密态回放、单向隐私推理与本作品方案置于同一能力矩阵中比较，以说明差异主要体现在动态剪枝、双向隐私、服务端权威快检与可复核控制面的联合成立，而非单一协议指标。

若进一步按技术路线划分，SecureML、GAZELLE、CrypTen 与 Delphi 更接近通用私有推理系统，Iron、BOLT、BumbleBee 与 NEXUS 更接近固定结构 Transformer 的协议优化路线，CipherPrune 则进一步把密态词元剪枝与近似降阶结合起来。与这些方案相比，本作品更偏向在既有 DynamicViT 训练主线上完成安全化改写，并将动态词元决策、双向隐私边界、服务端权威快检与审计证据统一纳入同一条执行链路。

因此，该方案的定位更接近“动态结构安全化 + 双向隐私 + 工程闭环”的综合方案，而非单纯追求某一项协议指标的最优值。这一路线的优势在于更便于复现、验证和部署迁移，但其结论也必须在既定数据集、bundle 口径和 2PC 运行边界内解读。

能力项	固定结构安全推理	明文剪枝 + 密态回放	单向隐私推理	本作品
输入数据隐私保护	✔	✘	🌕	✔
模型参数隐私保护	✔	✔	🌕	✔
动态剪枝能力保留	✘	🌕	✘	✔
剪枝决策留在密态域内	✘	✘	✘	✔
服务端权威快检	✘	✘	🌕	✔
协议异常输入守卫	✘	🌕	🌕	✔
审计链与可追溯	✘	🌕	✘	✔
浏览器演示闭环	✘	✘	🌕	✔

✔ 支持

🌕 部分支持

✘ 不支持

图 5-1 本作品与常见方案能力边界对比图

5.5 当前局限性

5.5.1 方法与实验边界

当前展示后端采用单进程 Uvicorn/FastAPI 原型承载长耗时 SPU 调用，主动断连感知、任务取消和队列化调度能力仍不完整。因此，现有并发守卫只能说明原型级请求门控有效，不能等价替代生产级拒绝服务防护。

当前仓内已固化端到端通信量与算子级代理基准证据，但尚未单独落盘医疗动态安全剪枝链路的阶段级时延剖面。因此，本作品尚不能给出更细粒度的阶段性能分解。

金融任务当前仅覆盖 8 条边界压力样本，这足以支撑迁移稳定性验证，但不足以单独构成第二应用主线。相应地，本作品在金融场景上的结论仅限于边界压力测试，而不应扩展为完整业务验证。

当前安全模型主要建立在两方半诚实假设上，不覆盖恶意参与方串谋、模型抽取和更强侧信道对抗场景。因此，本作品结论只在既定威胁模型下成立，不能直接外推到更强对抗设置。当前系统尚未系统评估输出结果本身可能带来的反推风险，因此输出侧隐私仍需在后续工作中单独建模与验证。这意味着本作品已证明输入侧与执行侧边界，但尚未穷尽输出侧风险。

SViT 等外部对照方案虽然在协议级性能方面表现较优，但其部署角色、信任假设和执行形态与本作品并不完全一致。因此，这类对照更适合用于技术定位，而不宜

被解读为同边界下的端到端优胜关系。

5.5.2 路线扩展与后续比较边界

当前动态安全剪枝链路已经证明 DynamicViT 安全化重写的可行性,但尚未在统一数据、统一硬件和统一执行边界下系统比较 Iron、BOLT、NEXUS、CipherPrune 等路线的全部收益差异。因此,本作品更适合作为面向医疗影像动态剪枝场景的工程基线,而不是对所有私有 Transformer 方案给出普适最优结论。

总体而言,本作品当前结论严格限定在指定数据集、冻结 bundle、2PC SPU 后端和半诚实不串谋威胁模型内。后续若要吸收密态渐进剪枝、非交互协议或多方设置下的潜在收益,还需要同步调整 keep-mask 生成策略、训练目标、近似算子、bundle 组织方式和部署验证链路。

首先,本作品当前文档中的“医院侧 - 模型提供侧 - 协调侧”属于业务部署角色,并不等同于安全多方计算中的“三个计算方”。当前实际密态计算仍运行在 P1/P2 两个计算节点构成的 2PC SPU 后端之上。若要将本作品框架扩展到 3PC (Three-Party Computation) 及以上,一方面需要在底层运行时将当前 2pc.json 所对应的两方 Cheetah 配置替换为新的多方协议后端;另一方面还需要同步改造 share 生成、比较协议封装、运行时配置、日志统计、错误恢复和部署编排等接口,使其从“面向两方节点”转为“面向 p 个计算方”的统一抽象。从复杂度角度看,若输入张量大小记为 S ,则在 p 方秘密分享下,输入方至少需要向其余 $(p - 1)$ 个计算方分发 share,输入分发开销可由 2PC 下的 $O(S)$ 增长为 $O((p - 1)S)$;若底层协议采用两两通信的全互联拓扑,则单轮逻辑通信链路数为 $\binom{p}{2} = O(p^2)$ 。因此,3PC 及以上扩展的主要价值在于降低对单一参与方的信任依赖、提升容错与部署灵活性,但其代价是更多的跨节点同步、更复杂的协议实现以及更高的系统通信组织成本。换言之,本作品框架在体系结构上具备向 3PC+ 扩展的接口基础,但当前作品的实证结论仍严格限定在两方 2PC 执行边界内。

最后,代码中虽保留了 `lut\_gelu\_interp` 等 LUT 激活钩子,但当前正式 bundle 与展示链路主要采用的是 `fixed\_square` 激活路径,LUT table 尚未形成稳定、可复现的正式部署实测链路。因此,LUT 型非线性近似只能作为未来工作候选方向,而不应写成本作品已经完成的正式结果。后续工作可进一步围绕离线预处理材料缓存、在线转换裁剪以及多方设置下的协议边界展开更细粒度分析,以形成更完整的协议层

性能模型。

第六章复现与参赛声明

6.1 最低可复现路径

表 6-1 系统最低可复现验证步骤

步骤	预估耗时	入口	预期输出
启动 Web 演示并加载一张本地医疗样本	3 - 8 分钟	<code>python3 tools/start_showcase_spu_demo.py --host 0.0.0.0 --port 7860</code>	浏览器在本地完成预处理与分片后返回分类结果、质量保障结论、审计摘要与控制面耗时
运行协议层异常输入验证	2 - 4 分钟	<code>python3 tools/showcase_protocol_fuzz.py --base-url http://127.0.0.1:7860</code>	生成 <code>`protocol_fuzz_evidence.json`</code> （或自定义输出文件），验证 multipart/JSON/截断请求体阻断
运行至少一项控制面守卫检查	2 - 6 分钟	<code>python3 tools/showcase_guard_stress.py --base-url http://127.0.0.1:7860 --checks duplicate_nonce</code>	验证重复 nonce、重复载荷或并发占满守卫
可选：更换样本重复复核	2 - 5 分钟	浏览器重新加载另一张本地样本；或再次运行 <code>python3 tools/showcase_guard_stress.py --base-url http://127.0.0.1:7860 --checks duplicate_nonce</code>	验证完整计算链路可重复执行，且审计摘要与控制面统计持续生成

注：耗时为在普通开发环境（单机，8 核 CPU，16GB 内存）下的经验范围，实际耗时可能因硬件性能与数据规模而异。

系统最低可复现验证步骤如表 6-1 所示。在模型目录与基础 Python 环境已就位的前提下，可在 10 - 20 分钟内完成最小闭环验证：先启动 Web 演示并在浏览器端加载一张本地医疗样本，确认系统在本地完成预处理与分片后返回分类、质量保障结论、审计摘要与控制面耗时；再运行协议层异常输入脚本与至少一项控制面守卫检查，验证异常输入阻断与并发/重放防护。完整命令与故障排查说明见 ``README_REPRODUCE.md``。

从复现层次看，当前仓库提供三档入口：本机展示样例用于验证浏览器预处理、分片生成与 SPU 分类闭环；协议层异常输入和控制面守卫验证用于复核边界阻断、资源状态回落与重放/并发防护；真实两方部署骨架用于演练医院侧、模型提供方与

协调侧的分角色部署。三档入口共享同一 bundle、阈值口径与证据链文件，但目标不同：它们证明系统可运行、关键边界可验证、部署迁移有骨架，并不等同于已经完成生产环境集成。

6.2 数据、模型与权重合规声明

本作品对数据、模型结构与权重文件采用分层合规声明，具体边界如表 6-2 所示。数据层面，报告与演示不包含真实、未经授权的患者隐私信息，医疗结果仅用于研究验证与展示，不作为临床诊断依据；模型层面，公开仓库保留模型结构、运行脚本、图件、报告生成链与必要 bundle 元数据；权重层面，完整权重和原始验证数据不随 Git 默认分发，需在授权范围内单独提供。

因此，本作品区分“可公开交付物”和“受授权约束资源”：前者包括代码、报告、图件、验证脚本、公开证据、运行入口和必要 bundle 元数据，便于核查系统结构与结果口径；后者包括完整权重、原始数据集以及任何可能恢复患者信息或受限模型资产的内容，不作为默认可再分发资产。正文中的精度、通信量和鲁棒性结论均建立在授权可用的模型权重、指定验证集和当前冻结 bundle 之上，不意味着第三方在未获得授权时可无条件复现实验数值。

表 6-2 数据、模型与权重合规边界说明

对象	当前做法	合规边界
测试数据	仅使用声明范围内的验证数据与压力样本	不对外分发未经授权的原始敏感数据
模型结构代码	随仓保留并可复现运行链	遵循仓内第三方来源与修改映射说明
模型权重	默认不随 Git 完整分发，仅保留 bundle 元数据与加载入口	需由维护方按授权边界单独提供
第三方预训练/ 上游组件	按原许可证与来源映射保留	不对上游本体主张排他性知识产权

注：本声明仅适用于本次提交的原型系统；任何商业使用需单独获得相关数据与模型的授权许可。

6.3 第三方许可、原创边界与技术公开范围

本作品的知识产权声明按“上游底座、基于 Apache 2.0 的本地适配、团队原创部分”三层边界撰写。报告只描述仓内已实际完成的物理映射，不对未存在的许可证分发项或未落地的文件头声明作超前表述。第三方组件许可与原创边界的物理映射关系如表 6-3 所示。

本作品的安全计算底座基于 SecretFlow SPU，并按事实保留上游 Apache License 2.0 许可证文本和本地修改说明；当前核对到的上游分发物未发现独立 NOTICE 文件，因此不虚构额外分发项。第三方许可汇总由`THIRD\_PARTY.md`说明，许可证文本索引由`licenses/README.md`提供，SPU vendored 部分的本地修改入口由`spu\_vendored/MODIFICATIONS.md`记录。通过许可证索引、修改说明和报告正文互相印证，可以明确区分上游底座、本地适配、团队原创贡献和受授权限制的实验资产，降低提交和公开时的口径不一致风险。

本作品的原创贡献主要集中在动态控制流映射策略、浏览器工作线程控制面、服务端权威快检、前后端审计链闭环、演示程序集成以及报告与图件生成链；完整模型权重和数据集按授权边界单独管理，不在公开仓库中默认再分发。

表 6-3 第三方组件许可与原创边界物理映射

物理映射项	仓内落点	当前事实
上游许可证文本	spu_vendored/LICENSE	已保留 Apache License 2.0 文本
vendored 变更总说明	spu_vendored/MODIFICATIONS.md	已列出当前交付版明确引用的本地适配文件
修改文件头声明 1	spu_vendored/libspu/spu.proto	文件头含密捷项目本地修改声明
修改文件头声明 2	spu_vendored/libspu/mpc/cheetah/arithmetic.h	文件头含密捷项目本地供应链适配声明
修改文件头声明 3	spu_vendored/libspu/mpc/cheetah/arithmetic.cc	文件头含密捷项目本地供应链适配声明
修改文件头声明 4	spu_vendored/libspu/mpc/cheetah/protocol.cc	文件头含密捷项目本地供应链适配声明
第三方来源总表	THIRD_PARTY.md	已保留 DynamicViT/OpenBumbleBee 来源与许可证映射

注：主要修改文件已在文件头或配套修改说明中标注本地修改来源；本作品不对上游开源组件主张任何排他性知识产权。

参考文献

- [1] Shamir A. How to Share a Secret[J]. Communications of the ACM, 1979, 22(11):612-613. DOI:10.1145/359168.359176.
- [2] Yao A C. Protocols for Secure Computations[C]//23rd Annual Symposium on Foundations of Computer Science(FOCS). 1982:160-164. DOI:10.1109/SFCS.1982.38.
- [3] Goldreich O, Micali S, Wigderson A. How to Play Any Mental Game[C]//Proceedings of the 19th Annual ACM Symposium on Theory of Computing(STOC). 1987:218-229. DOI:10.1145/28395.28420.
- [4] Gentry C. Fully Homomorphic Encryption Using Ideal Lattices[C]//Proceedings of the 41st Annual ACM Symposium on Theory of Computing(STOC). 2009:169-178. DOI:10.1145/1536414.1536440.
- [5] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale[C]//International Conference on Learning Representations(ICLR). 2021.
- [6] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]//Advances in Neural Information Processing Systems(NeurIPS). 2017.
- [7] Hendrycks D, Gimpel K. Gaussian Error Linear Units(GELUs)[EB/OL]. arXiv:1606.08415, 2016.
- [8] Ba J L, Kiros J R, Hinton G E. Layer Normalization[EB/OL]. arXiv:1607.06450, 2016.
- [9] Rao Y, Zhao W, Liu B, Lu J, Zhou J, Hsieh C J. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification[C]//Advances in Neural Information Processing Systems(NeurIPS). 2021.
- [10] Demmler D, Schneider T, Zohner M. ABY: A Framework for Efficient Mixed-Protocol Secure Two-Party Computation[C]//Network and Distributed System Security Symposium(NDSS). 2015.
- [11] Beaver D. Efficient Multiparty Protocols Using Circuit Randomization[C]//Advances in Cryptology—CRYPTO 1991. LNCS 576. Springer, 1991:420-432. DOI:10.1007/3-540-46766-1_34.

- [12] Cabrero-Holgueras J, Pastrana S. SoK: Privacy-Preserving Computation Techniques for Deep Learning[J]. Proceedings on Privacy Enhancing Technologies, 2021, 2021(4):139-162. DOI:10.2478/popets-2021-0064.
- [13] Mohassel P, Zhang Y. SecureML: A System for Scalable Privacy-Preserving Machine Learning[C]//2017 IEEE Symposium on Security and Privacy(SP). 2017:19-38.
- [14] Liu J, Juuti M, Lu Y, Asokan N. Oblivious Neural Network Predictions via MiniONN Transformations[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security(CCS). 2017:619-631.
- [15] Juvekar C, Vaikuntanathan V, Chandrakasan A. GAZELLE: A Low Latency Framework for Secure Neural Network Inference[C]//27th USENIX Security Symposium(USENIX Security). 2018:1651-1669.
- [16] Riazi M S, Samragh M, Chen H, Laine K, Lauter K, Koushanfar F. XONN: XNOR-based Oblivious Deep Neural Network Inference[C]//28th USENIX Security Symposium(USENIX Security). 2019:1501-1518.
- [17] Mishra P, Lehmkuhl R, Srinivasan A, Zheng W, Popa R A. Delphi: A Cryptographic Inference Service for Neural Networks[C]//29th USENIX Security Symposium(USENIX Security). 2020:2505-2522.
- [18] Knott B, Venkataraman S, Hannun A, et al. CrypTen: Secure Multi-Party Computation Meets Machine Learning[C]//Advances in Neural Information Processing Systems(NeurIPS). 2021.
- [19] 马敏, 付钰, 黄凯, 贾潇风. 基于秘密共享的轻量级隐私保护 ViT 推理框架[J]. 通信学报, 2024, 45(4):27-38. DOI:10.11959/j.issn.1000-436x.2024025.
- [20] Hao M, Li H, Chen H, et al. Iron: Private Inference on Transformers[C]//Advances in Neural Information Processing Systems(NeurIPS). 2022.
- [21] Pang Q, Zhu J, Möllering H, Zheng W, Schneider T. BOLT: Privacy-Preserving, Accurate and Efficient Inference for Transformers[C]//2024 IEEE Symposium on Security and Privacy(SP). 2024.
- [22] Lu W, et al. BumbleBee: Secure Two-party Inference Framework for Large Transformers[C]//Network and Distributed System Security Symposium(NDSS). 2025.

- [23] Zhang J, Yang X, He L, et al. Secure Transformer Inference Made Non-interactive[C]//Network and Distributed System Security Symposium(NDSS). 2025.
- [24] Chen T, Bao H, Huang S, et al. THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption[C]//Findings of the Association for Computational Linguistics: ACL 2022. 2022:3510-3520. DOI:10.18653/v1/2022.findings-acl.277.
- [25] Li D, Shao R, Wang H, et al. MPCFormer: Fast, Performant and Private Transformer Inference with MPC[C]//International Conference on Learning Representations(ICLR). 2023.
- [26] Luo J, Zhang Y, Zhang Z, et al. SecFormer: Fast and Accurate Privacy-Preserving Inference for Transformer Models via SMPC[C]//Findings of the Association for Computational Linguistics: ACL 2024. 2024:13333-13348. DOI:10.18653/v1/2024.findings-acl.790.
- [27] Liang Y, Ge C, Tong Z, Song Y, Wang J, Xie P. EViT: Expediting Vision Transformers via Token Reorganizations[C]//International Conference on Learning Representations(ICLR). 2022.
- [28] Yin H, Vahdat A, Alvarez J M, Mallya A, Kautz J, Molchanov P. A-ViT: Adaptive Tokens for Efficient Vision Transformer[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR). 2022:10809-10818. DOI:10.1109/CVPR52688.2022.01054.
- [29] Bolya D, Fu C-Y, Dai X, Zhang P, Feichtenhofer C, Hoffman J. Token Merging: Your ViT But Faster[C]//International Conference on Learning Representations(ICLR). 2023.
- [30] Zeng W, et al. MPCViT: Searching for Accurate and Efficient MPC-Friendly Vision Transformer with Heterogeneous Attention[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision(ICCV). 2023.
- [31] Zhang Y, Xue J, Zheng M, et al. CipherPrune: Efficient and Scalable Private Transformer Inference[C]//International Conference on Learning Representations(ICLR). 2025.
- [32] Touvron H, Cord M, Douze M, Massa F, Sablayrolles A, Jégou H. Training data-efficient image transformers & distillation through attention[C]//Proceedings of

- the 38th International Conference on Machine Learning(ICML). 2021.
- [33] Hinton G, Vinyals O, Dean J. Distilling the Knowledge in a Neural Network[EB/OL]. arXiv:1503.02531, 2015.
- [34] Ma J, et al. SecretFlow-SPU: A Performant and User-Friendly Framework for Privacy-Preserving Machine Learning[C]//USENIX Annual Technical Conference(USENIX ATC). 2023.
- [35] Batcher K E. Sorting Networks and Their Applications[C]//Proceedings of the AFIPS Spring Joint Computer Conference. 1968:307-314.
- [36] Deng J, Dong W, Socher R, et al. ImageNet: A Large-Scale Hierarchical Image Database[C]//IEEE Conference on Computer Vision and Pattern Recognition(CVPR). 2009:248-255. DOI:10.1109/CVPR.2009.5206848.
- [37] Masinter L. Returning Values from Forms: multipart/form-data[S]. RFC 7578. IETF, 2015.
- [38] National Institute of Standards and Technology. Secure Hash Standard(SHS)[S]. FIPS PUB 180-4. Gaithersburg, MD: NIST, 2015.
- [39] WHATWG. HTML Living Standard: Web Workers[EB/OL]. <https://html.spec.whatwg.org/dev/workers.html>, 2026-05-20.
- [40] Huang G, Liu Z, van der Maaten L, Weinberger K Q. Densely Connected Convolutional Networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition(CVPR). 2017.
- [41] Nair V, Hinton G E. Rectified Linear Units Improve Restricted Boltzmann Machines[C]//International Conference on Machine Learning(ICML). 2010:807-814.
- [42] Fawcett T. An Introduction to ROC Analysis[J]. Pattern Recognition Letters, 2006, 27(8):861-874. DOI:10.1016/j.patrec.2005.10.010.
- [43] Patra A, Schneider T, Suresh A, Yalame H. ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation[C]//30th USENIX Security Symposium(USENIX Security). 2021:2165-2182.

附录 A 关键代码实现

本附录围绕“离线训练准备 -> 冻结导出 -> 在线承接”的主线组织关键代码摘录。每一节均先给出代码位置，再说明其与正文主线的对应关系，最后保留最短必要代码片段。

A. 1 动态剪枝训练主线与保留率配置

该段代码对应离线训练阶段如何把 DynamicViT 的词元筛选语义显式写入训练入口。pruning_loc 决定三处剪枝层位，keep_rate 决定逐阶段 token 保留预算，并为后续部署阶段的 keep-mask 语义保留提供训练侧约束。

以下仅保留与正文论证直接相关的关键代码节选，已省略无关分支、重复检查与通用异常处理。

```

01 pruning_loc = [3, 6, 9]
02 keep_rate = [sparse_ratio[0], sparse_ratio[0] ** 2, sparse_ratio[0] ** 3]
03 model = VisionTransformerDiffPruning(
04     pruning_loc=pruning_loc,
05     token_ratio=keep_rate,
06     secure_static_depth=args.secure_static_train_depth,
07     secure_static_skip_pruning=args.secure_static_skip_pruning,
08 )
09 criterion = DistillDiffPruningLoss_dynamic(
10     teacher_model,
11     criterion,
12     cls_distill_weight=args.cls_distill_weight,
13     token_distill_weight=args.token_distill_weight,
14 )

```

A. 2 蒸馏补偿与部署对齐配置

这部分代码说明训练阶段不仅产出明文精度，还会记录`cls\_distill\_weight`、`token\_distill\_weight`、`secure\_static\_train\_depth`与`secure\_static\_skip\_pruning`等部署对齐参数，并由配对比较脚本回收同口径命令参数。

以下仅保留与正文论证直接相关的关键代码节选，已省略无关分支、重复检查与通用异常处理。

```

01 command_flags = parse_command_flags(command_path)
02 cls_weight = parse_float(command_flags.get("cls_distill_weight"), default=0.0)
03 token_weight = parse_float(command_flags.get("token_distill_weight"), default=0.0)
04 report = {
05     "configured_distill": {
06         "cls_distill_weight": command_flags.get("cls_distill_weight"),
07         "token_distill_weight": command_flags.get("token_distill_weight"),
08         "secure_static_train_depth":
09 command_flags.get("secure_static_train_depth"),
10         "secure_static_skip_pruning":
11 command_flags.get("secure_static_skip_pruning"),
    }
}

```

A. 3 阈值搜索、冻结导出与 bundle 组织

该部分代码体现离线训练结束后先搜索部署阈值，再冻结权重、写出参数快照和 manifest，最终形成可由在线 2PC 执行链路读取的正式 bundle。

以下仅保留与正文论证直接相关的关键代码节选，已省略无关分支、重复检查与通用异常处理。

```

01 def find_best_threshold(class1_prob, targets):
02     best = None
03     for threshold in candidate_thresholds(class1_prob):
04         acc, pred = accuracy_at_threshold(class1_prob, targets, threshold)
05         candidate = {
06             "threshold": float(threshold),
07             "accuracy": float(acc),
08             "pred_counts": [int(v) for v in torch.bincount(pred,
09 minlength=2).tolist()],
10         }
11         if best is None or candidate["accuracy"] > best["accuracy"]:
12             best = candidate
13     return best
14
15 export_state_path = output_dir / DEFAULT_BUNDLE_MODEL_STATE_NAME
16 torch.save(model_state, export_state_path)
17 (output_dir / "args_snapshot.json").write_text(json.dumps(args_snapshot, indent=2))
    (output_dir / "manifest.json").write_text(json.dumps(manifest, indent=2))

```

A. 4 在线安全执行承接接口

这一组代码对应在线阶段如何承接离线冻结语义。浏览器 Worker 负责分片与摘要，服务端在进入 SPU 前执行权威快检，SPU 侧则以 encoded-key 和 index-tracking bitonic sort 构造 keep-mask。

以下仅保留与正文论证直接相关的关键代码节选，已省略无关分支、重复检查与通用异常处理。

```

01 def _secure_build_keep_decision(score, prev_decision_2d, keep_count):
02     N = int(score.shape[1])
03     active_before = prev_decision_2d.squeeze(-1) > 0
04
05     epsilon = 1e-6
06     index_vals = jnp.arange(N, dtype=score.dtype) * epsilon
07     encoded_key = score - index_vals
08     encoded_masked = jnp.where(active_before, encoded_key, float("-inf"))
09
10     padded_count = 1
11     while padded_count < N:
12         padded_count *= 2
13     if padded_count > N:
14         neg_inf = jnp.full((int(score.shape[0]), padded_count - N), float("-inf"),
15 dtype=score.dtype)
16         sortable = jnp.concatenate([encoded_masked, neg_inf], axis=1)
17     else:
18         sortable = encoded_masked
19
20     sorted_keys, sorted_indices = _bitonic_sort_desc_with_indices(sortable)
21     top_k_indices = sorted_indices[:, :keep_count]
22     pos = jnp.arange(N, dtype=jnp.int32)
23     keep_mask = jnp.any(top_k_indices[:, :, None] == pos[None, None, :], axis=1)
24     keep_mask = keep_mask[:, :N] & active_before
    return keep_mask[:, :, None]

```


附录 B 鲁棒性验证

表 B-1 各类异常场景的攻击特征与系统拦截层级明细

异常场景	攻击载荷特征	首个拦截层级	兜底层级
① HTTP / MIME 层攻击			
Chunked 编码绕过	Transfer-Encoding: chunked 绕过 Content-Length	HTTP Body Gate	不适用
超大 Content-Length	明文体积 >5 MiB, 触发 413 强制阻断	Content-Length Gate	TCP 强制阻断
重复字段名	multipart 重复字段	Multipart Precheck	MIME 结构校验
Boundary 局出	构造过多分片, 放大 MIME 解析树	Multipart Precheck	MIME 结构校验
嵌套 multipart	将 share part 伪装成 multipart/mixed	Multipart Precheck	MIME 结构校验
超长 JSON	client_quality_summary 超过 JSON 长度限制	JSON Length Gate	严格 JSON 解码钩子
字段集漂移	追加未声明字段, 验证精确字段集约束	Multipart Precheck	精确字段集门
Boundary 参数混淆	boundary 带空白与引号变体	Content-Type / Boundary Parser	Raw Multipart 预检
畸形 Part Header	缺失合法头格式, 验证 header parser	Multipart Header Parser	MIME 结构校验
Header 空字节注入	在 part header 中注入 NUL/控制字符	Multipart Header Parser	MIME 结构校验
非空 Epilogue	closing boundary 后追加无效数据	Multipart Precheck	MIME 结构校验
② JSON 元数据攻击			
异常 JSON 字符集	UTF-16LE 元数据, 违反 UTF-8 编码约束	Strict UTF-8 JSON 解码	JSON 数值钩子
截断请求体	提前断流, 验证 streaming body reader	Streaming Body Reader	不适用
③ Payload 重放攻击			
重复 Nonce 并发重放	同 nonce 并发多发, 验证幂等与重放防护	Nonce Cache	Payload 指纹去重门
同载荷换 Nonce	相同 share 指纹 + 新 nonce, 验证 payload 去重	Payload Cache	IP Inflight 配额门
同 IP 并发占满	单一来源地址并发冲击已通过前置校验的执行配额	IP Inflight Guard	全局 Inflight 守卫
④ 请求生命周期攻击			
短窗限频	同 IP 短时间空载请求探测限流门	Sliding Window Limiter	不适用

说明：首个拦截层级为最先阻断异常的组件；兜底层级为在前置未拦截时的最终防线。

表 B-2 异常场景的系统返回处置、资源状态与证据来源明细

序号	异常场景	返回/处置	系统状态（观测值变化）	证据来源
HTTP / MIME 结构异常	1 Chunked 编码绕过	HTTP 400/ transfer_encoding_not_supported	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A1
	2 超大 Content-Length	HTTP 413/ payload_too_large	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A2
	3 重复字段名	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +152$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A3
	4 Boundary 局出	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A3
	5 嵌套 multipart	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +1024$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A4
	6 超长 JSON	HTTP 400/ invalid_control_plane_payload	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +5120$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A6
	7 字段集源移	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +192$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A7
	8 Boundary 参数混淆	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A8
	9 畸形 Part Header	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A9
	10 Header 空字节注入	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A10
	11 非空 Epilogue	HTTP 400/ malformed_multipart_precheck_failed	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A11
元数据异常	12 异常 JSON 字符集	HTTP 400/ invalid_control_plane_payload	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +6604$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A12
	13 截断请求体	HTTP 400/ truncated_body	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +0$ KiB; $\Delta Thr = +0$	协议异常输入验证记录 /A13
重放与伪造	14 重复 Nonce 并发重放	duplicate_nonce $\times 3$; 200 $\times 1$	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +30632$ KiB; $\Delta Thr = +0$	控制面守卫验证记录/G1
	15 同载荷换 Nonce	首发 200; 复发 duplicate_payload	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +16032$ KiB; $\Delta Thr = +0$	控制面守卫验证记录/G2
	16 同 IP 并发占满	200 $\times 2$; busy_retry_later $\times 2$	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +47004$ KiB; $\Delta Thr = +0$	控制面守卫验证记录/G3
资源与速率 防护	17 短窗限频	invalid_content_length $\times 6$; rate_limited_ip $\times 2$	$\Delta FD = +0$; $\Delta Sock = +0$; $\Delta RSS = +128$ KiB; $\Delta Thr = +0$	控制面守卫验证记录/G4

注： ΔFD 表示文件描述符数量变化， $\Delta Sock$ 表示套接字文件描述符数量变化， ΔRSS 表示常驻内存变化， ΔThr 表示线程数变化，四项指标均按异常处理后观测值减去基线观测值计算。系统状态指标 ΔFD 、 $\Delta Sock$ 、 ΔRSS 、 ΔThr 均为异常处理完成后 10 秒的观测值与基线值的差值；所有证据文件均已留存于仓内 tools/evidence 目录。

附录 C 前端展示

本附录用于补充说明本作品的前端展示界面与交互流程。结合本作品“离线训练与部署准备”以及“在线隐私推理展示”的总体主线，前端界面分为管理员端与用户端两部分。其中，管理员端对应当前已实现的训练控制台，用于训练任务创建、模型资产管理、结果证据回溯与环境配置；用户端对应面向实际展示场景的推理与报告界面，用于说明病例上传、隐私推理过程提示与辅助诊断报告回传流程。相关界面如图 C-1 至图 C-10 所示。

C.1 管理员端界面展示

本作品管理员端作为训练与交付闭环的可视化控制入口，主要承担训练任务提交、任务状态查看、日志回溯、模型版本管理、结果证据展示以及训练服务器连接配置等功能。与传统静态展示页不同，该界面能够承接真实训练任务调度与结果整理流程，从而形成“训练任务 - 阈值校准 - 部署导出 - 结果回溯”的统一管理入口。

图 C-1 展示了管理员端登录页及训练服务器连接配置区域。该界面支持在本机训练服务与远端训练服务器之间进行切换，并可在登录前完成目标后端地址测试与保存，从而为后续训练任务调度提供统一入口。登录后也可进行连接切换。



图 C-1 管理员端登录与训练服务器连接配置界面

图 C-2 展示了管理员端总览看板。该页面集中呈现当前训练队列状态、正式主线模型指标、GPU 运行环境以及训练输出目录等关键信息，便于在展示过程中快速说明系统当前运行状态与主线模型交付情况。

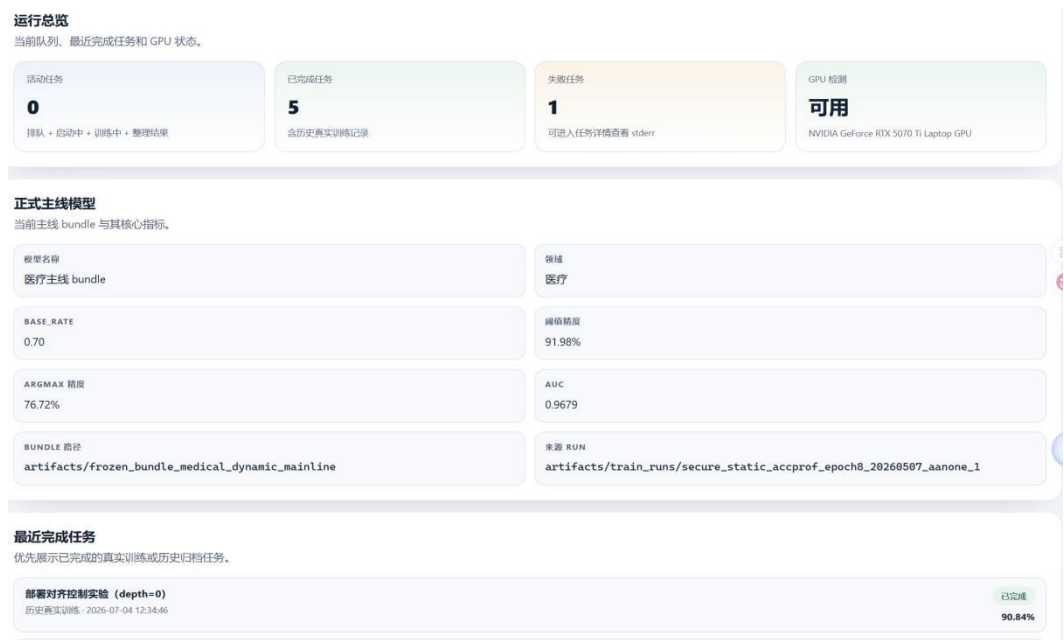


图 C-2 管理员端总览看板界面

图 C-3 展示了训练任务创建界面。管理员可在该页面选择训练预设方案或手动填写参数，并配置训练集路径、验证集路径、基础保留率、蒸馏权重、训练轮次、设备类型及部署导出选项等参数，用于发起真实训练任务。

新建训练任务

默认采用现有训练预设，可局部改写关键参数。

任务名称: 现场展示训练任务

启动方式: 主线预设

预设模板: 医疗动态主线

动态剪枝、蒸馏补偿与冻结导出主线。

训练集路径	data\medical_train_placeholder	验证集路径	data\medical_val_placeholder
训练轮次	8	批大小	32
数据线程	4	训练设备	cuda
基础保留率	0.7	secure_static_train_depth	12
分类蒸馏权重	1	token 蒸馏权重	0.02
☒ 启用动态剪枝		☒ 启用安全友好算子	
☒ 完成后导出 bundle		finetune 检查点	artifacts/frozen_bundle_secure_static_depth12_uniform_fix

图 C-3 管理员端训练任务创建界面

图 C-4 展示了任务详情页。该页面用于跟踪训练任务当前阶段、查看命令链、读取标准输出与错误日志，并回溯训练权重、阈值搜索结果和部署导出文件位置，是证明本作品前端能够承接真实训练流程的重要界面。

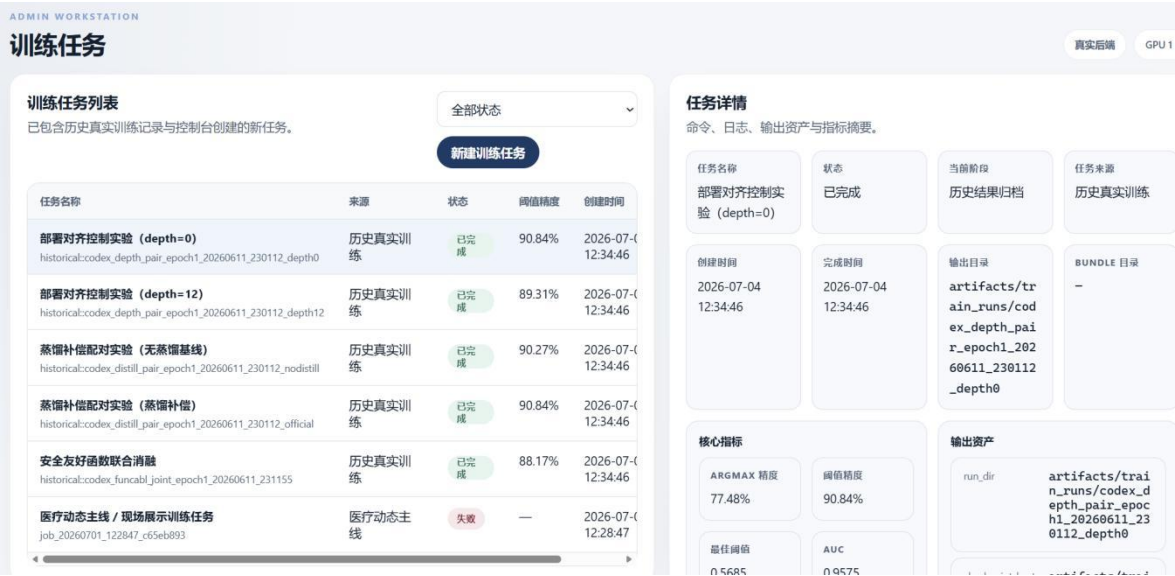


图 C-4 管理员端任务详情与日志界面

图 C-5 展示了模型资产管理与结果证据展示界面。前者用于展示已训练模型、部署 bundle 及其来源路径，图 C-6 用于回溯正式指标对应的原始结果文件，从而将报告中的主线指标与实际 JSON 结果文件建立直接对应关系。



图 C-5 管理员端模型资产界面

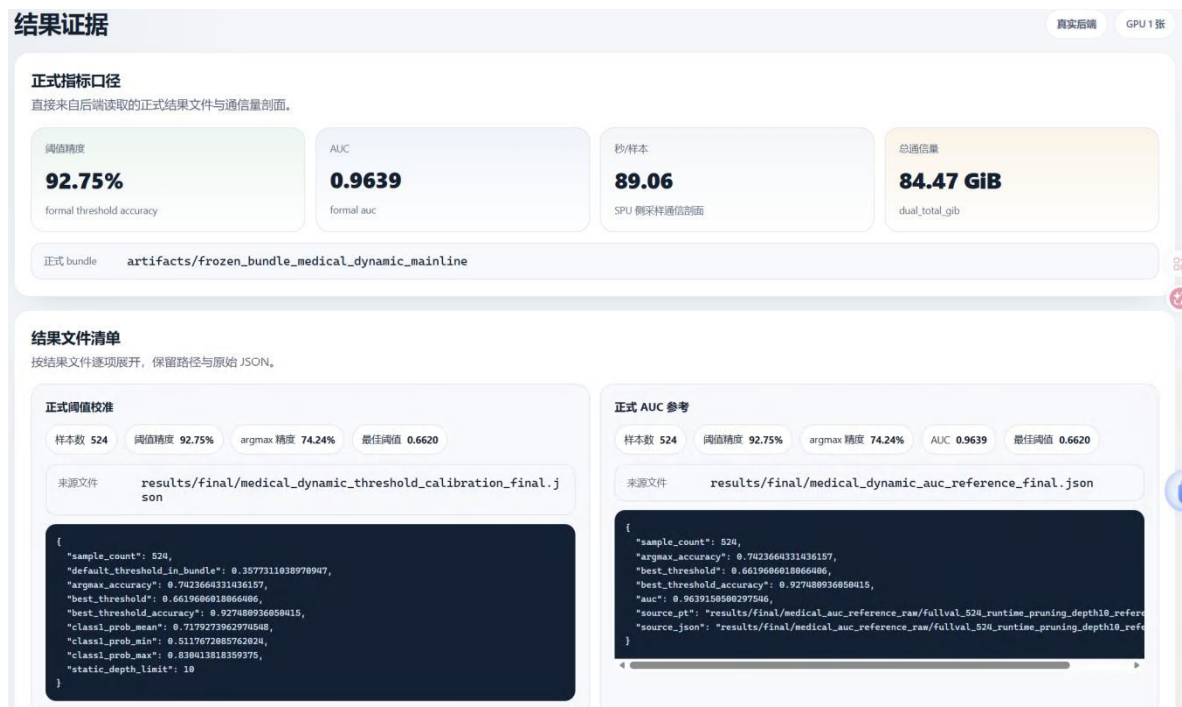


图 C-6 管理员端结果证据界面

C. 2 用户端界面展示

本作品用户端面向隐私推理展示场景，重点体现病例上传、推理进度反馈、结果回传与标准化辅助诊断报告呈现等功能。其作用作为在线展示环节的交互承接界面，用于说明用户侧如何完成推理发起、状态查看与结果接收流程。

图 C-7 展示了用户端首页。该页面提供新建隐私推理、历史报告查看等功能入口，并对近期任务状态和隐私保护提示进行集中展示。



图 C-7 用户端首页界面

图 C-8 展示了作品剪枝特色与执行拓扑。用于更好的理解本作品。



图 C-8 作品剪枝与执行概览界面

图 C-9 展示了用户端隐私推理创建界面。用户可在该页面完成病例影像上传，并根据界面提示进入后续隐私处理与推理流程。



图 C-9 用户端隐私推理创建界面

图 C-10 展示了推理过程中的阶段性进度提示。该界面用于说明本作品在用户侧的流程反馈机制，包括本地处理、服务端预检、密态计算与结果返回等关键阶段。



图 C-10 用户端推理进度展示界面

图 C-11 展示了用户端报告详情页。该界面以标准化报告形式呈现病例信息、分类结论、置信度区间、生成时间及备注内容，是本作品结果交付链路在用户侧的最终承接页面。



图 C-11 用户端辅助诊断报告详情界面

C.3 附录小结

综上，前端界面是对本作品训练管理、结果证据回溯和在线推理交互流程的可视化承接。其中，管理员端侧重离线训练与部署准备阶段的管理闭环，用户端侧重在线推理展示与报告回传流程，二者共同构成本作品完整的交互展示链路。